Week 03: Edges & Contours

From "looks like edges" to **precise edge detection**

From scattered pixels to **meaningful shapes**



Recap of last week



Example: box filter

$$g[\cdot,\cdot]$$

$$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$



Slide credit: David Lowe (UBC)

Image filtering

$g[\cdot,\cdot]$ $\frac{1}{9}$

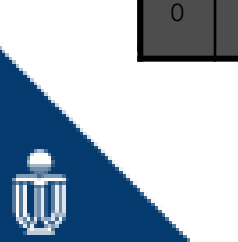
1	1	1
1	1	1
1	1	1

$f[.,.]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[.,.]$

$$h[m,n] = \sum_{k,l} g[k,l]\, f[m+k, n+l]$$



Credit: S. Seitz

Image filtering

$g[\cdot,\cdot]$ $\frac{1}{9}$

1	1	1
1	1	1
1	1	1

$f[.,.]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[.,.]$

	0	10							

$$h[m,n] = \sum_{k,l} g[k,l]\, f[m+k, n+l]$$

Credit: S. Seitz

Image filtering

$g[\cdot,\cdot]$ $\frac{1}{9}$

1	1	1
1	1	1
1	1	1

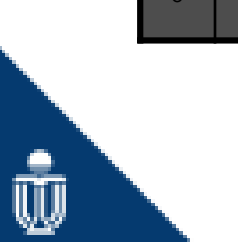
$f[.,.]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[.,.]$

	0	10	20						

$$h[m,n] = \sum_{k,l} g[k,l]\, f[m+k,n+l]$$



Credit: S. Seitz

Image filtering

$g[\cdot,\cdot]$ $\frac{1}{9}$

1	1	1
1	1	1
1	1	1

$f[.,.]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[.,.]$

	0	10	20	30					

$$h[m,n] = \sum_{k,l} g[k,l]\, f[m+k,n+l]$$

Credit: S. Seitz

Image filtering

$g[\cdot,\cdot]$ $\frac{1}{9}$

1	1	1
1	1	1
1	1	1

$f[.,.]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[.,.]$

	0	10	20	30	30				

$$h[m,n] = \sum_{k,l} g[k,l]\, f[m+k,n+l]$$

Credit: S. Seitz

Image filtering

$g[\cdot,\cdot]$ $\frac{1}{9}$

1	1	1
1	1	1
1	1	1

$f[.,.]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[.,.]$

	0	10	20	30	30				
				?					

$$h[m,n] = \sum_{k,l} g[k,l]\, f[m+k,n+l]$$

Credit: S. Seitz

Image filtering

$g[\cdot,\cdot]$ $\frac{1}{9}$

1	1	1
1	1	1
1	1	1

$f[.,.]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[.,.]$

	0	10	20	30	30				
						?			
				50					

$$h[m,n] = \sum_{k,l} g[k,l]\, f[m+k,n+l]$$

Credit: S. Seitz

Image filtering

$$g[\cdot,\cdot]\ \frac{1}{9}\begin{bmatrix}1&1&1\\1&1&1\\1&1&1\end{bmatrix}$$

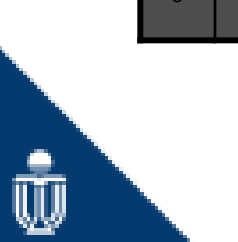
$f[.,.]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[.,.]$

	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	60	90	90	90	60	30	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	10	20	30	30	30	30	20	10	
	10	10	10	0	0	0	0	0	

$$h[m,n]=\sum_{k,l} g[k,l]\ f[m+k,n+l]$$

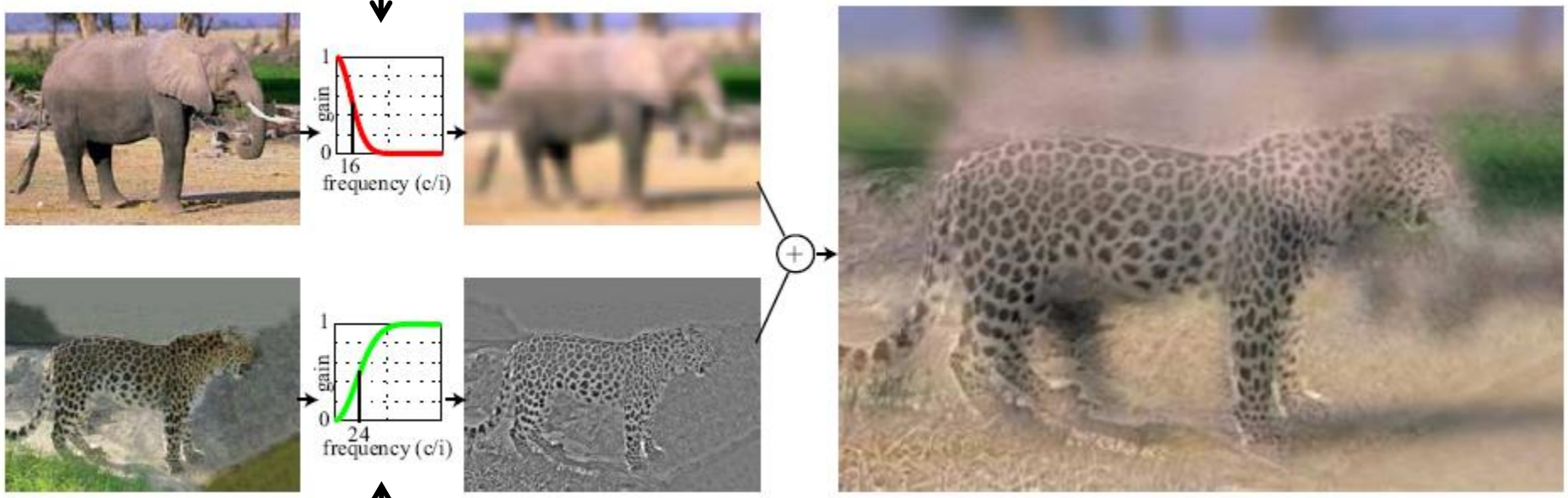


Credit: S. Seitz

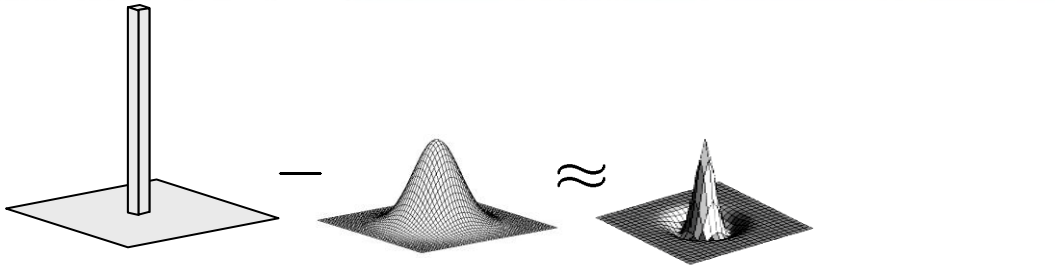
Application: Hybrid Images

A. Oliva, A. Torralba, P.G. Schyns, "Hybrid Images," SIGGRAPH 2006

Gaussian Filter!



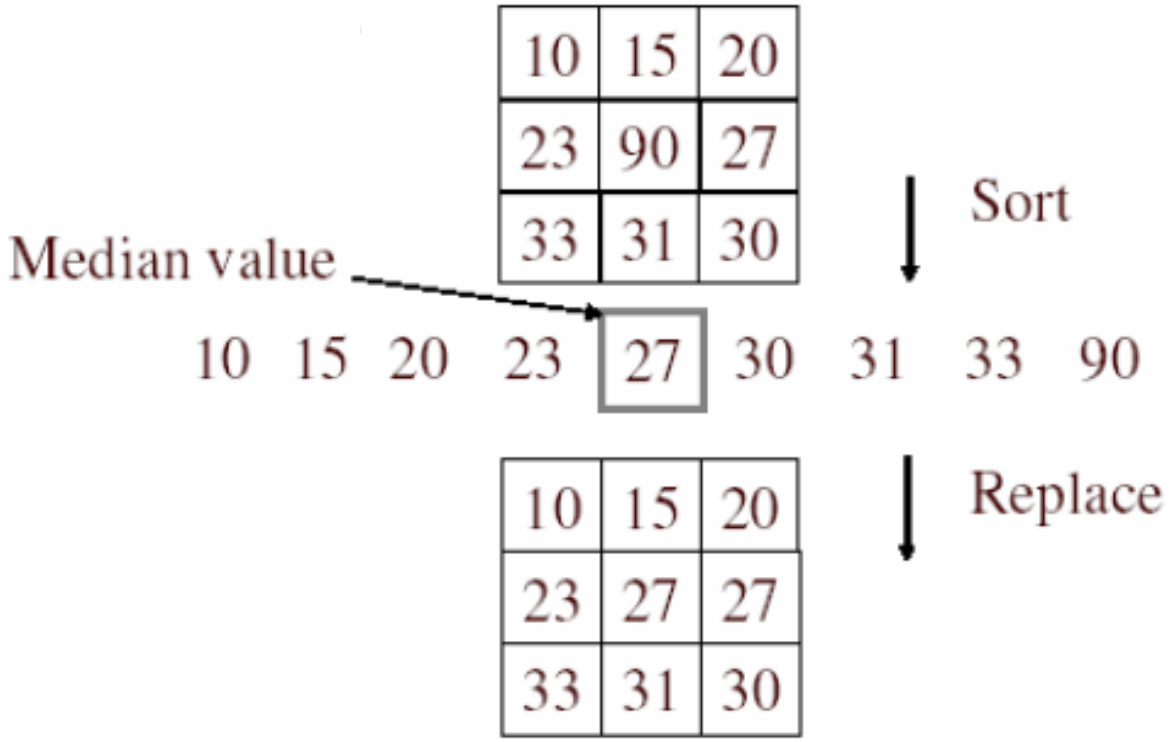
Laplacian Filter!



unit impulse — Gaussian ≈ Laplacian of Gaussian

Alternative idea: Median filtering

- A **median filter** operates over a window by selecting the median intensity in the window



- Is median filtering linear?

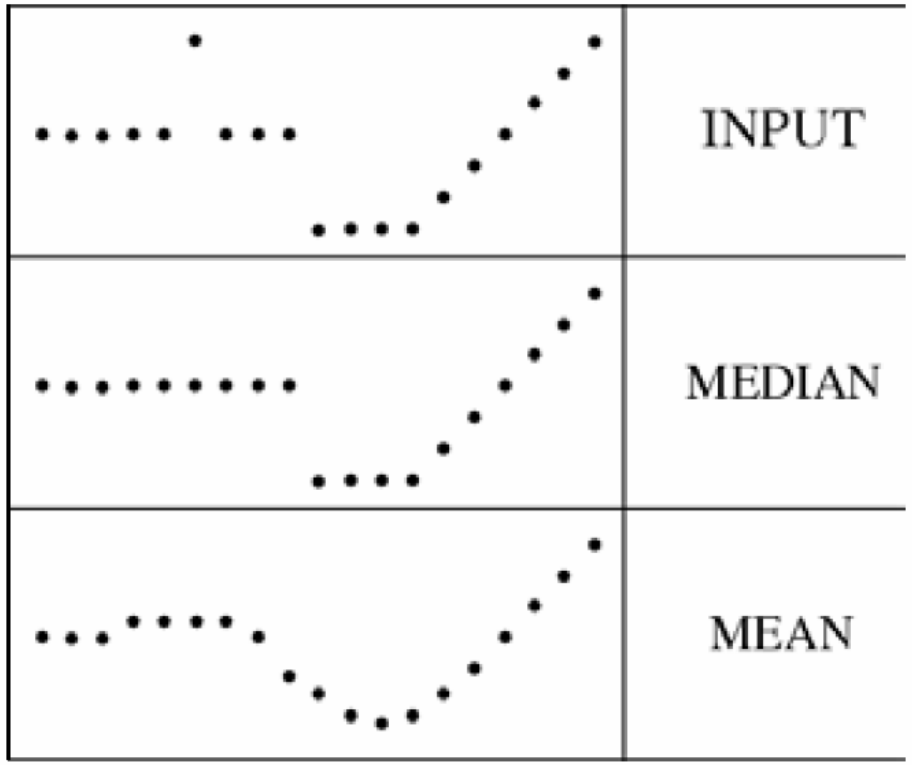


Source: K. Grauman

Median filter

- What advantage does median filtering have over Gaussian filtering?
  - Robustness to outliers

filters have width 5 :

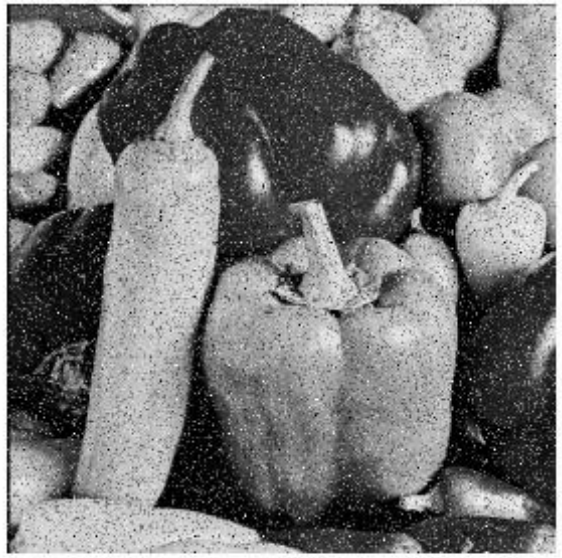


Source: K. Grauman

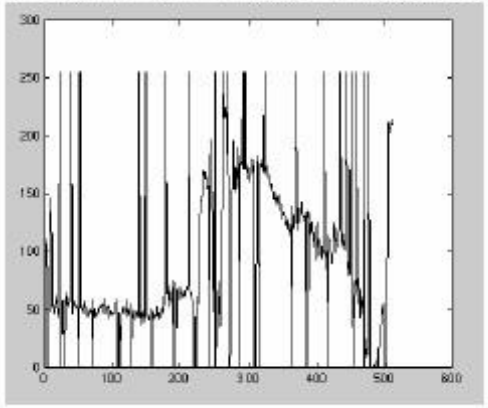
Median filter

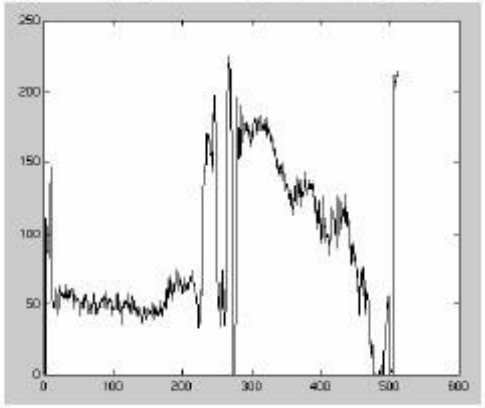
Salt-and-pepper noise

Median filtered









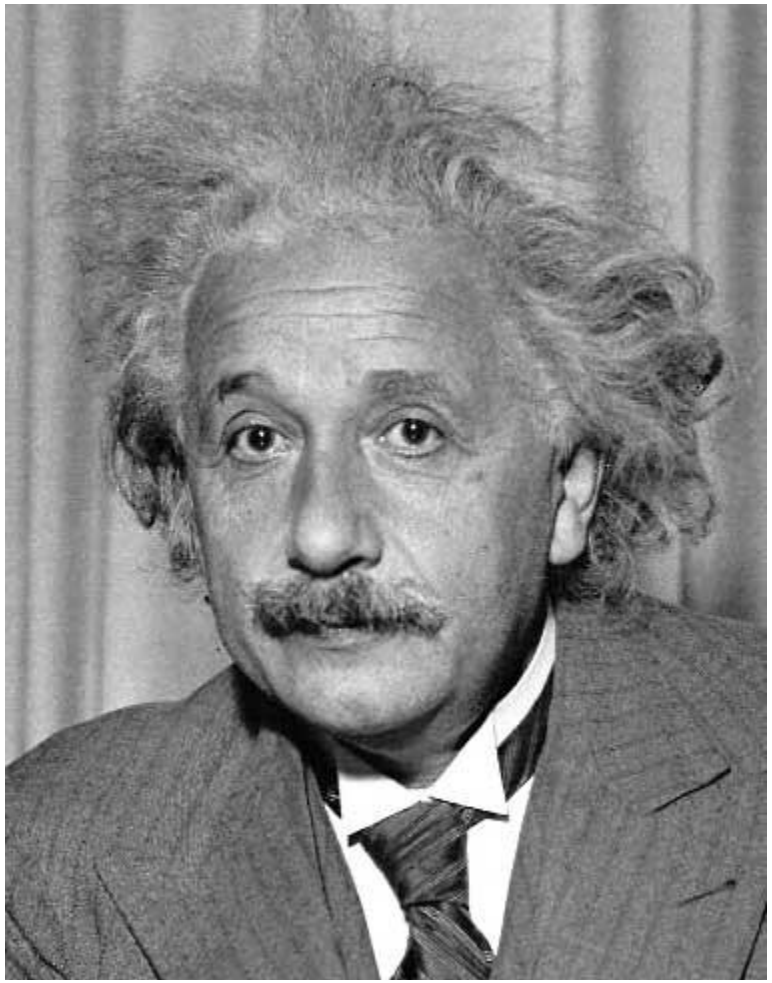
Python: scipy.ndimage.median_filter (image, size)

Source: M. Hebert

Template matching

- Goal: find  in image

- Main challenge: What is a good similarity or distance measure between two patches?



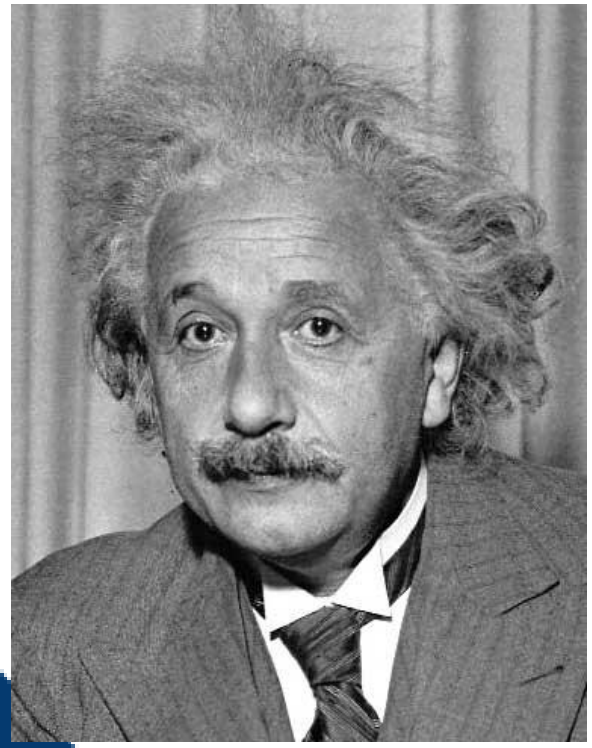


Matching with filters

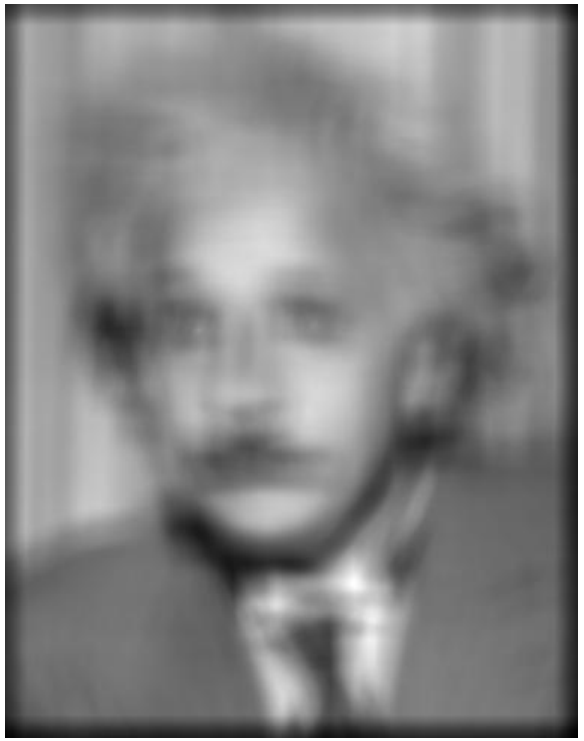
- Goal: find  in image
- Method 0: filter the image with eye patch

$$h[m,n] = \sum_{k,l} f[k,l]\ \ im[m+k, n+l]$$

im = image
f = filter



Input



Filtered Image

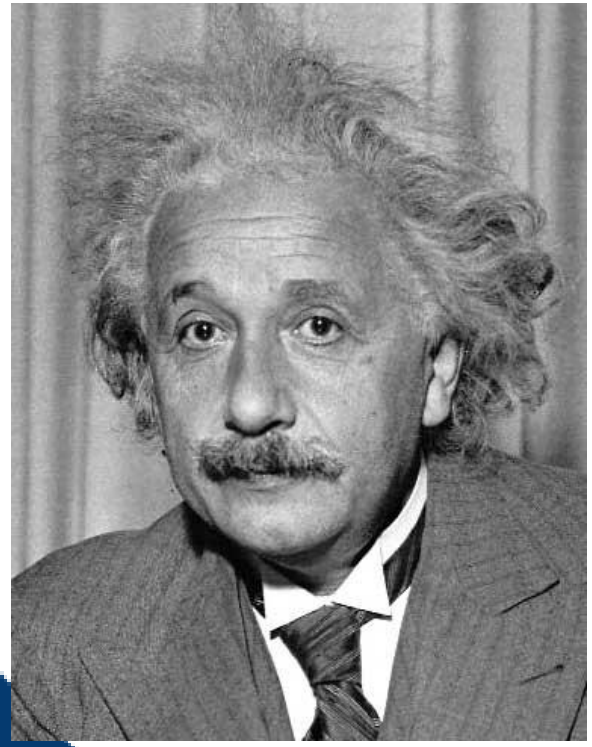
What went wrong?

Matching with filters

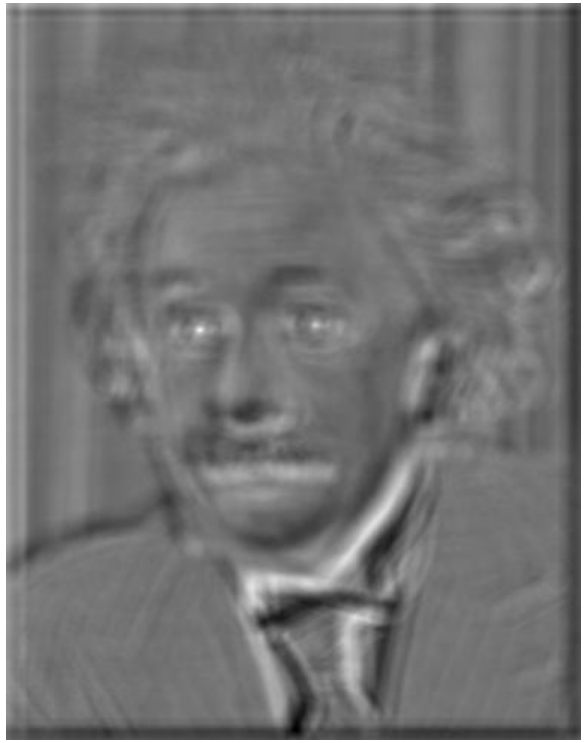
- Goal: find  in image
- Method 1: filter the image with zero-mean eye

$$h[m,n] = \sum_{k,l} (f[k,l] - \bar{f})\, im[m+k, n+l]$$

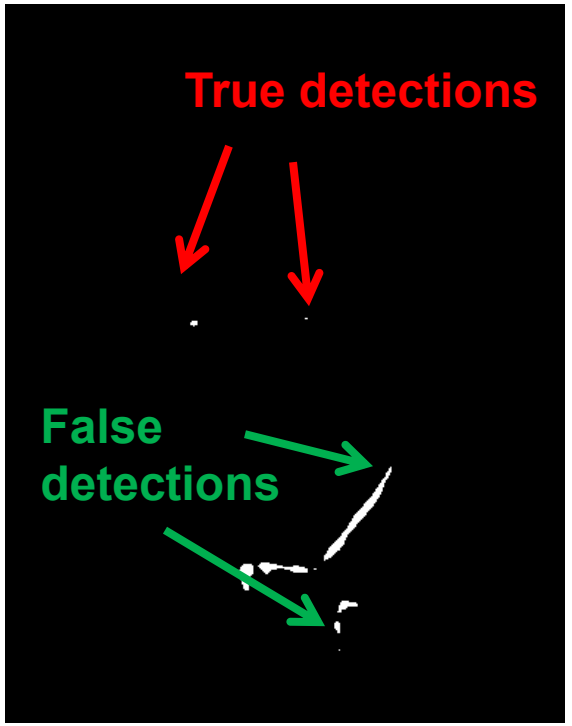
mean of filter f



Input



Filtered Image (scaled)



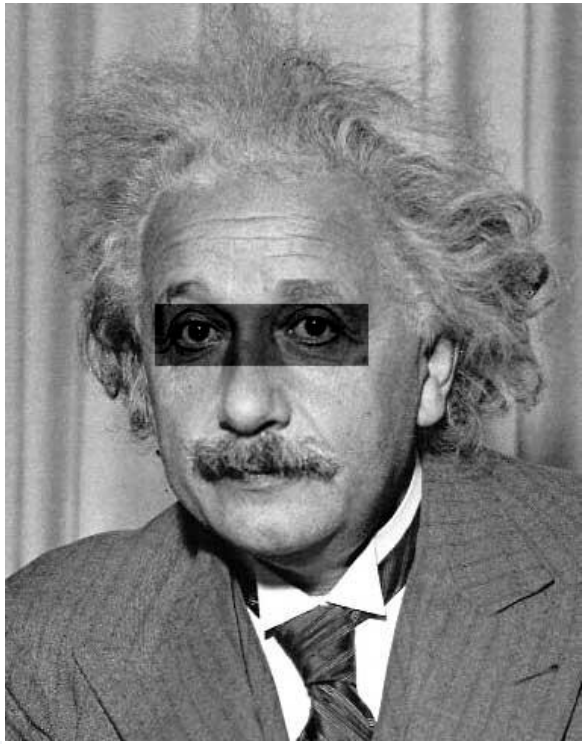
Thresholded Image

Matching with filters

- Goal: find  in image
- Method 2: Sum of Square Distance (SSD)

$$h[m,n] = \sum_{k,l} (f[k,l] - \ im[m+k, n+l])^2$$

What's the potential downside of SSD?



Input



1- sqrt(SSD)

Sensitive to intensity shift

Matching with filters

- Goal: find  in image
- Method 3: Normalized cross-correlation

mean template

mean image patch

$$h[m,n] = \frac{\sum_{k,l}(f[k,l] - \bar{f})(\, im[m+k, n+l] - \overline{im_{m,n}})}{\left(\sum_{k,l}(f[k,l] - \bar{f})^2 \sum_{k,l}(\, im[m+k, n+l] - \overline{im_{m,n}})^2\right)^{0.5}}$$

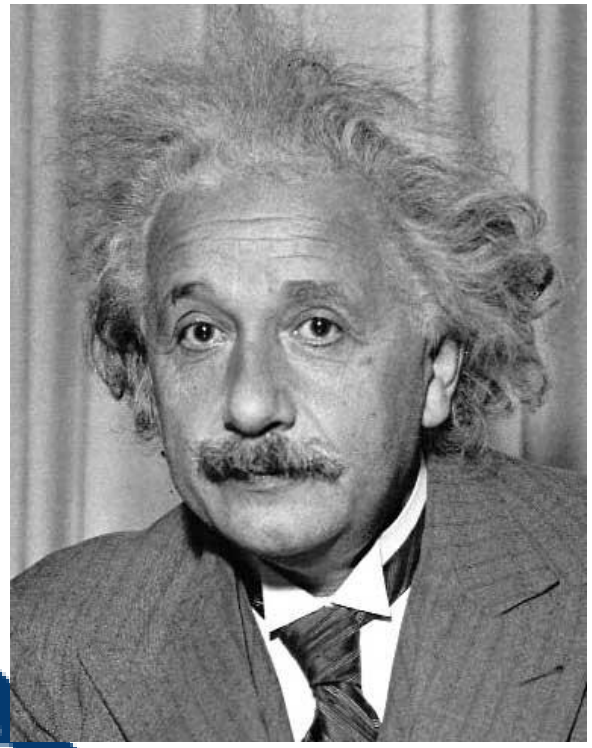
(divide by product of standard deviations of template and image patch)



Python: `cv2.matchTemplate(im,template,cv2.TM_CCOEFF_NORMED)`

Matching with filters

- Goal: find  in image
- Method 3: Normalized cross-correlation



Input



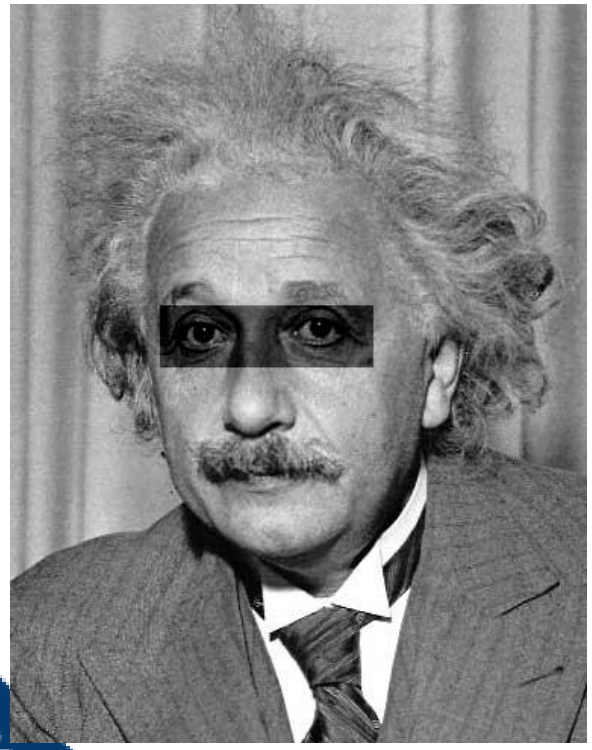
Normalized X-Correlation



Thresholded image

Matching with filters

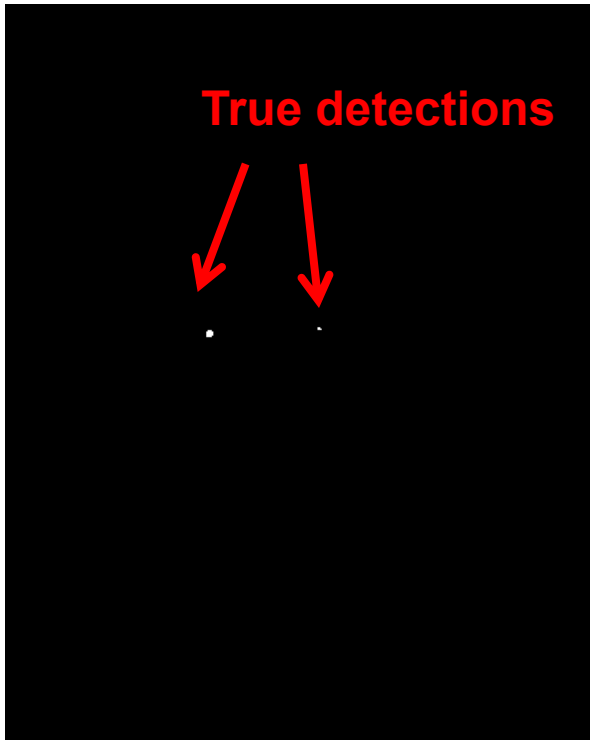
- Goal: find  in image
- Method 3: Normalized cross-correlation



Input



Normalized X-Correlation



Thresholded Image

The Coin Counting Challenge

How many coins? Can a computer count them?



The Challenge

How Many Coins?

- You can count them by eye
- But what if there were **1,000**?
- Or **10,000**?

Goal: By the end of today, we'll make the computer count them automatically.



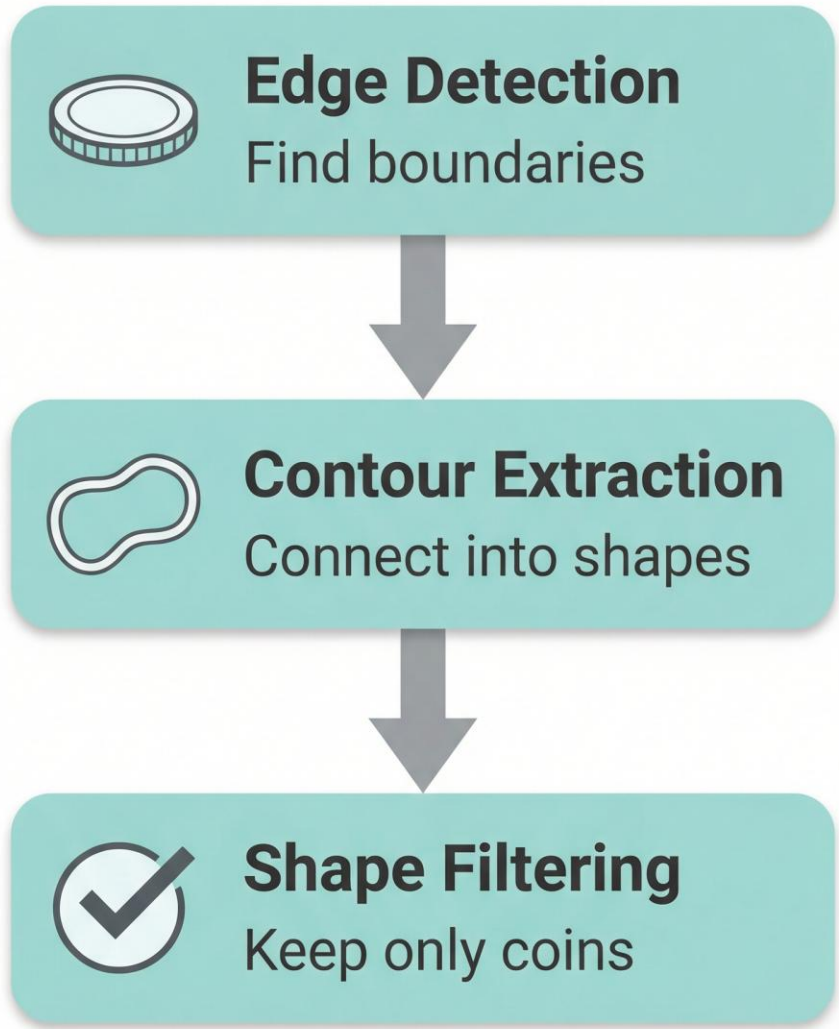
What Does the Computer Need to Do?

Breaking Down the Problem

1. **Find the boundary** of each coin
2. **Separate** individual coins
3. **Count** them

This requires:

- **Edge detection** → find boundaries
- **Contour extraction** → connect into shapes



Let's Try What We Know

Attempt: High-Frequency + Contour Detection

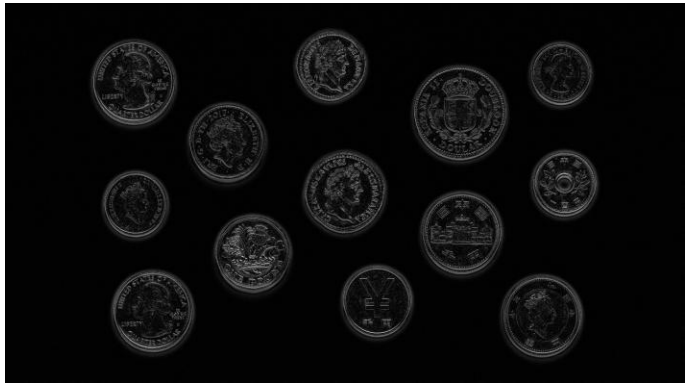
We already know how to extract "edges" from Week 2:

```
high_freq = coins − GaussianBlur(coins)
```

Then find contours on the high-frequency image...

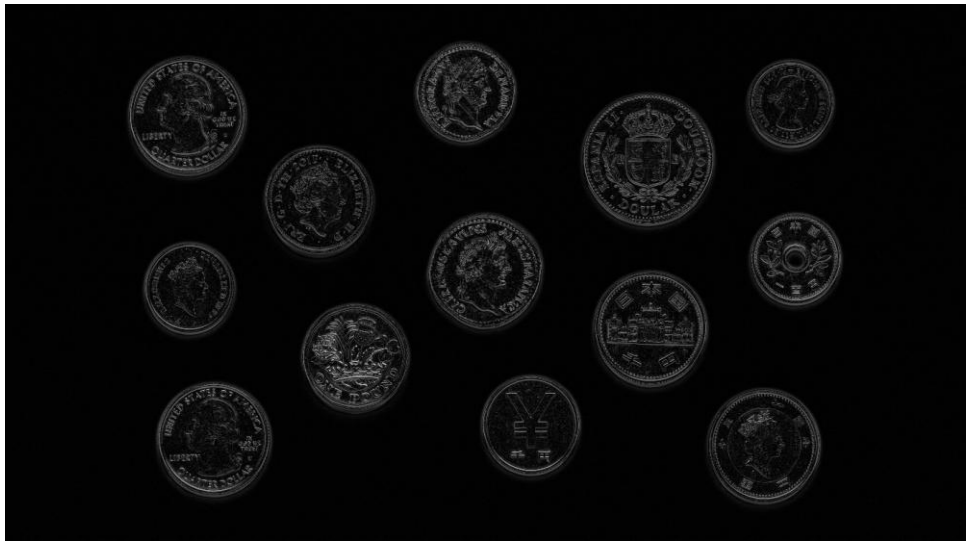


High Frequency ≈ Edges?

Original	Gaussian Blurred	High Frequency
		
Sharp details	Low-pass: smooth	**Looks like edges!** But is it precise?

Will it work?

High-Frequency + Contours = FAILURE

High-Frequency Image	Contour Detection Result
	
Noisy, disconnected edges	Expected 12 coins → got **200+ contours** ✗

Why Did It Fail?

Three Problems with High-Frequency Edges

Problem	Description
Disconnected	Edges break at many points — not continuous
Too much noise	Random variations create false "edges"
Too thick	Edges span 5–10 pixels wide, not precise

The high-frequency image gives us "the feeling of edges," but it cannot tell us **which exact pixel** is the edge.



Today's Journey

> **"From fuzzy edge feelings → precise edge positions"**

Start	Learn	Apply
High-freq fails	Sobel & Canny	Count coins correctly!
≈ edges	= edges	→ contours → count



Part 3

The Math Behind Edges

Derivatives, Gradients, and Why They Matter



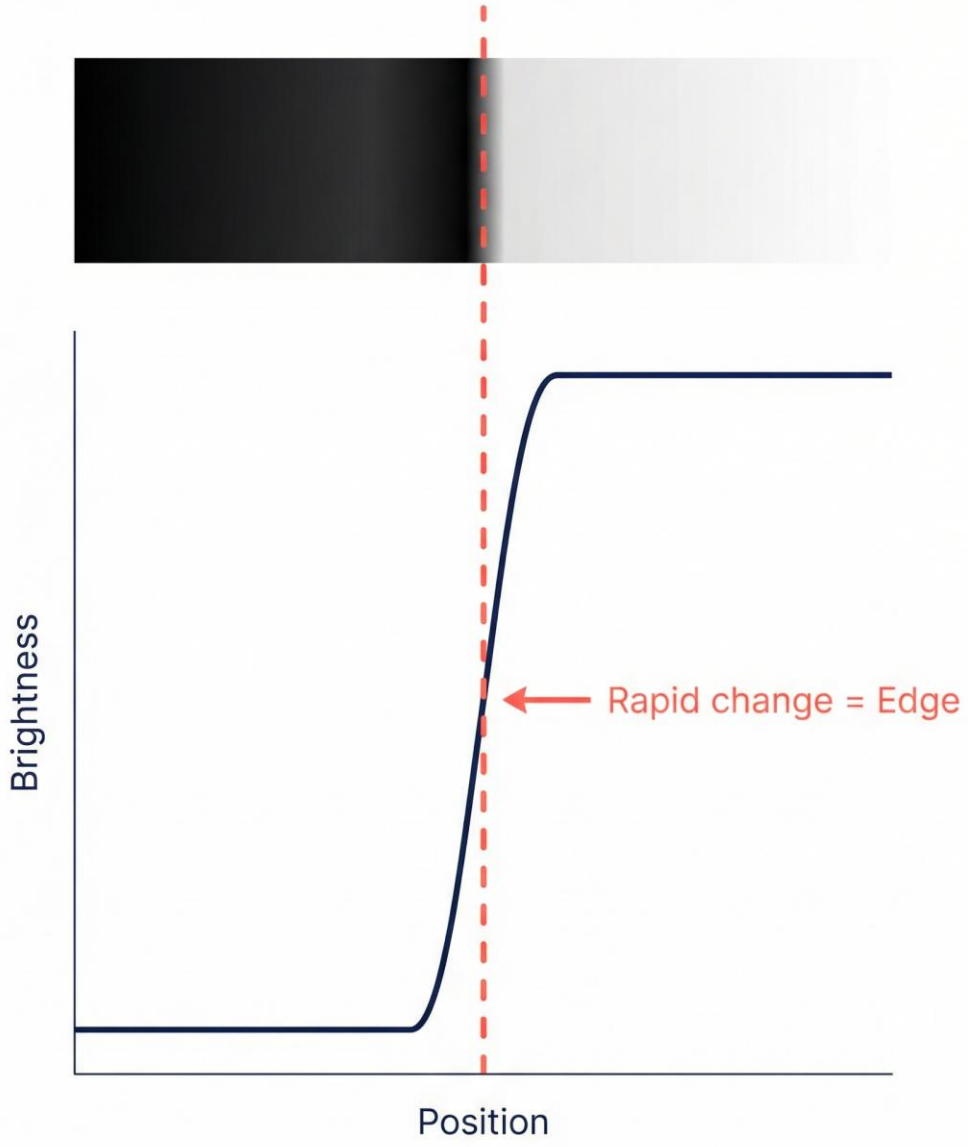
What Is an Edge?

Intuition

An **edge** is where **brightness changes dramatically**.

- Object vs background boundary
- Light vs shadow
- Different textures or colors

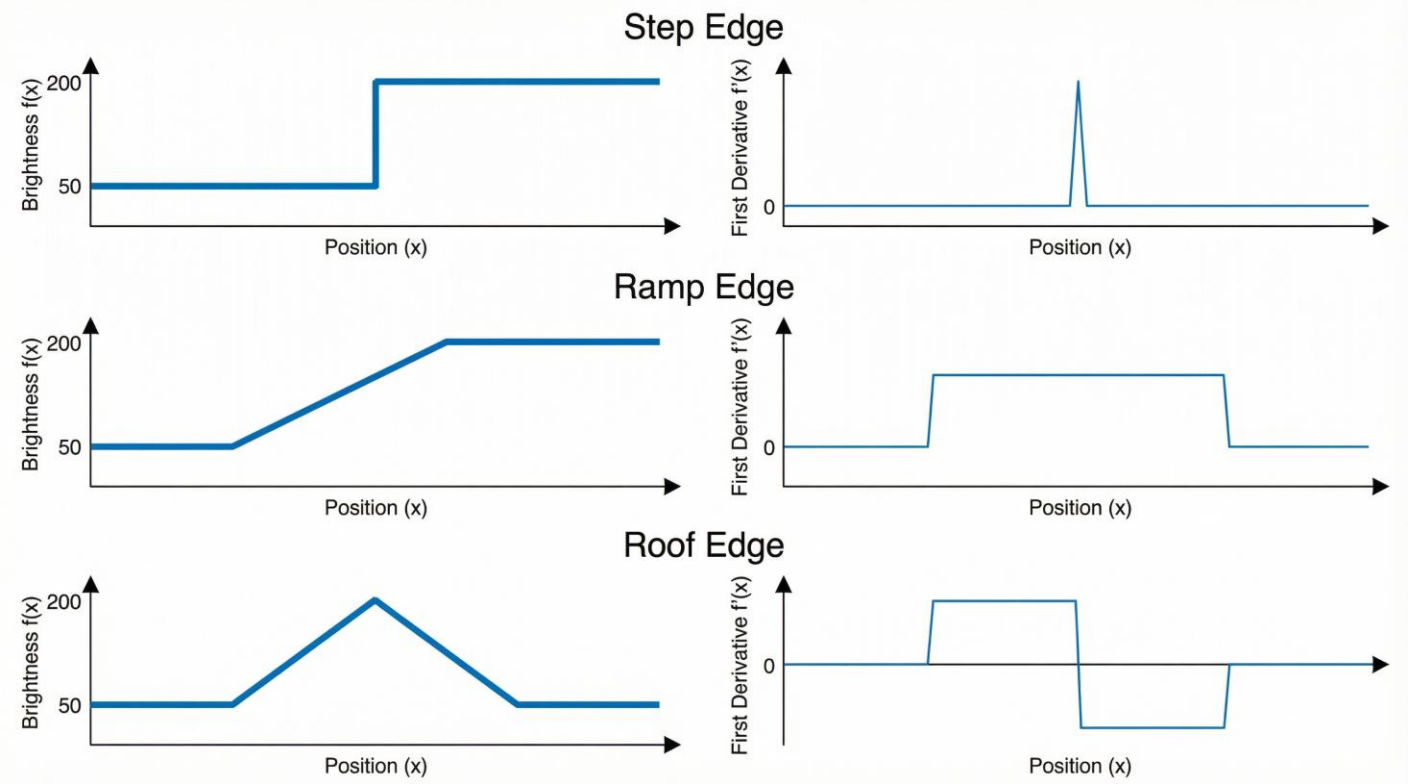
Mathematically: large **gradient magnitude**.



Types of Edges

Type	Real-World Example
Step Edge	Object boundary
Ramp Edge	Shadow boundary
Roof Edge	Thin line / wire

Each type produces a different **derivative** signature (see diagram →)



Change = Derivative

From Calculus to Images

In 1D, the **derivative** tells us how fast a function is changing:

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

- Large derivative → rapid change → **edge!**
- Zero derivative → no change → flat region



Discrete Image Derivatives

From Continuous to Discrete

Images are discrete — pixel values at integer positions. We approximate:

$$f'(x) \approx f(x+1) - f(x)$$

No calculus needed — just subtraction!



2D Images Need Gradients

The Gradient Vector

An image $I(x, y)$ is 2D, so we need partial derivatives in both directions:

$$\nabla I = \left(\frac{\partial I}{\partial x}, \frac{\partial I}{\partial y} \right) = (G_x, G_y)$$

- G_x: rate of change in the **horizontal** direction
- G_y: rate of change in the **vertical** direction



Gradient: Magnitude and Direction

Two Key Quantities

Gradient Magnitude — How strong is the edge?

$$|\nabla I| = \sqrt{G_x^2 + G_y^2}$$

Gradient Direction — Which way does brightness increase?

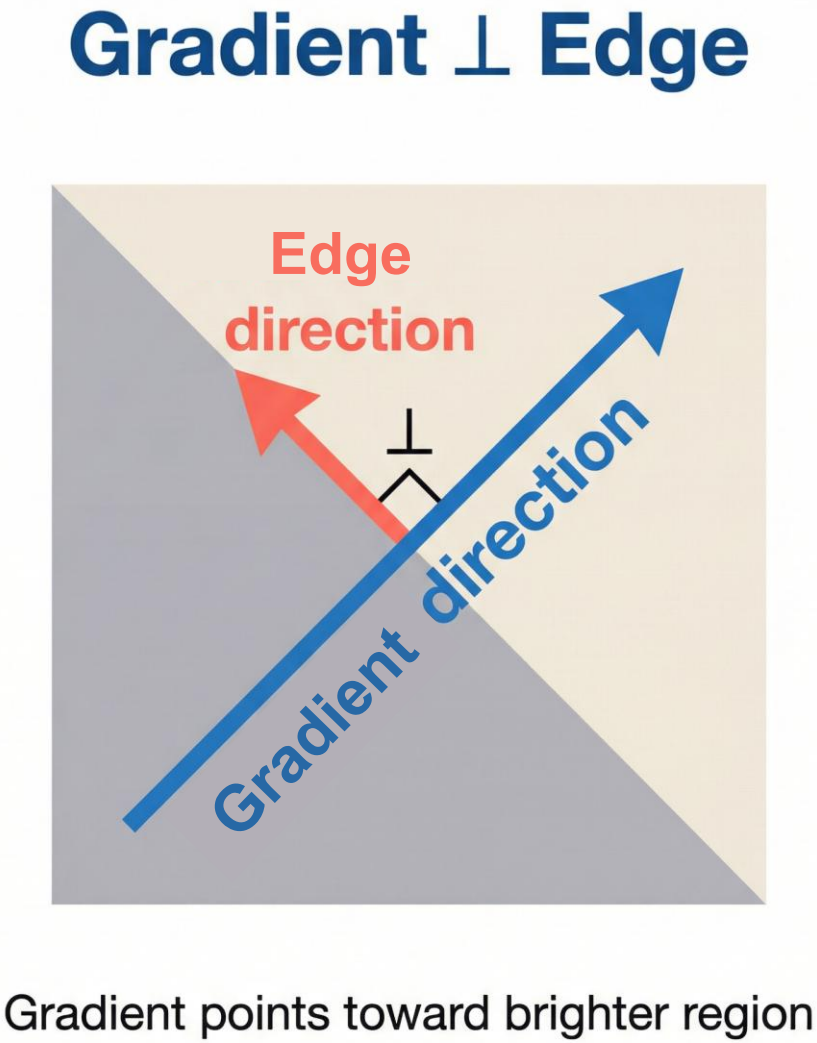
$$\theta = \arctan\left(\frac{G_y}{G_x}\right)$$



Key Insight: Gradient ⊥ Edge

A Crucial Property

- The **gradient direction** points toward steepest brightness increase
- The **edge direction** runs along the boundary
- Therefore: **gradient ⊥ edge**



Gradient points toward brighter region

The Sobel Operator

Computing Gradients with Convolution



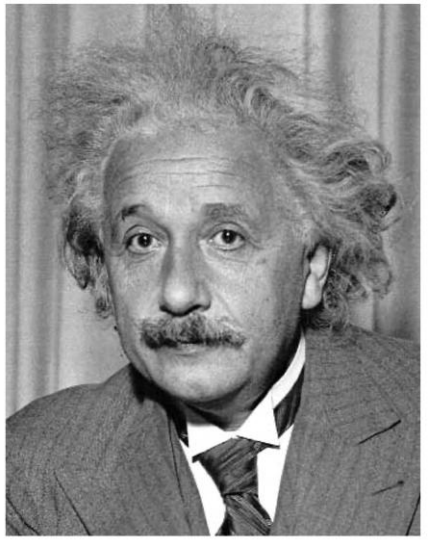
Convolution Can Compute Derivatives!

The Big Connection

Last week: convolution does **blurring, sharpening**...

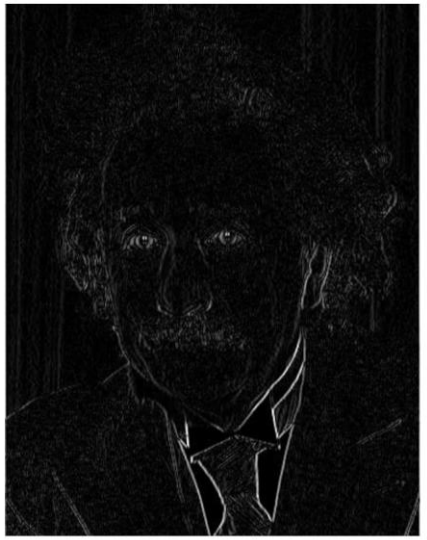
This week: convolution computes **derivatives!**

The **Sobel operator** is a specially designed derivative kernel.

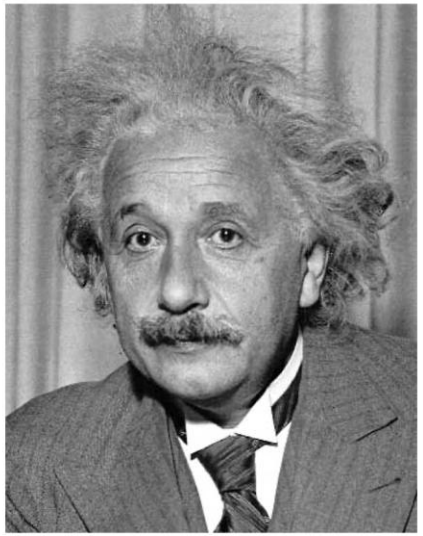


1	0	-1
2	0	-2
1	0	-1

Sobel

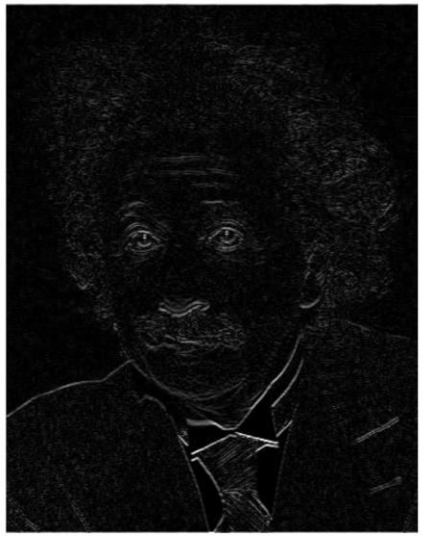


Vertical Edge
(absolute value)



1	2	1
0	0	0
-1	-2	-1

Sobel



Horizontal Edge
(absolute value)

Sobel X Kernel — Detects Vertical Edges

$$\text{Sobel}_X = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

- Computes the **horizontal** gradient G_x
- Detects **vertical** edges (where brightness changes left → right)
- Negative on left, positive on right → measures left-right difference



Sobel Y Kernel — Detects Horizontal Edges

$$\text{Sobel}_Y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

- Computes the **vertical** gradient G_y
- Detects **horizontal** edges (where brightness changes top → bottom)
- Negative on top, positive on bottom → measures top-bottom difference



Numerical Example: Setup

A 3×3 pixel patch with a **vertical edge** (dark left, bright right):

$$\text{Image} = \begin{bmatrix} 10 & 10 & 100 \\ 10 & 10 & 100 \\ 10 & 10 & 100 \end{bmatrix}$$

Applying Sobel X to the center pixel — multiply element-wise and sum...



Numerical Example: Edge Detected!

$$\text{Image} = \begin{bmatrix} 10 & 10 & 100 \\ 10 & 10 & 100 \\ 10 & 10 & 100 \end{bmatrix} \qquad \text{Sobel}_X = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

$$G_x = (-1)(10) + 0(10) + 1(100) + (-2)(10) + 0(10) + 2(100)$$
$$+(-1)(10) + 0(10) + 1(100)$$
$$G_x = -10 + 100 - 20 + 200 - 10 + 100 = \mathbf{360}$$

Large positive value → strong vertical edge!



Numerical Example: No Edge

A **uniform** region (no change):

$$\text{Image} = \begin{bmatrix} 100 & 100 & 100 \\ 100 & 100 & 100 \\ 100 & 100 & 100 \end{bmatrix}$$

$$G_x = (-1)(100) + 1(100) + (-2)(100) + 2(100) + (-1)(100) + 1(100) = \mathbf{0}$$

Zero → no edge here. The Sobel kernel correctly identifies flat regions.



🤔 Quick Question

Why does Sobel *X* detect *vertical* edges?

Isn't that confusing?

Answer: Sobel X computes the derivative in the **X (horizontal) direction**.

If brightness changes horizontally, there must be a **vertical** boundary.

Think: the X in Sobel X refers to the **direction of change**, not the edge orientation.



Why Sobel Beats Simple Difference

Feature	Simple Difference	Sobel
Pixels used	2 pixels only	3×3 neighborhood
Smoothing	None	Built-in (center weight = 2)
Noise sensitivity	High	Lower



Sobel's Design Insight

Differentiation + Smoothing in One Kernel

Sobel = **differentiation** + **smoothing** combined!

$$\text{Sobel}_X = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \otimes \begin{bmatrix} -1 & 0 & 1 \end{bmatrix}$$

Smooth vertically, differentiate horizontally — both in a single convolution.



Sobel in OpenCV

```
import cv2
import numpy as np

# Compute gradients (use CV_64F to keep negative values!)
sobelx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
sobely = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)

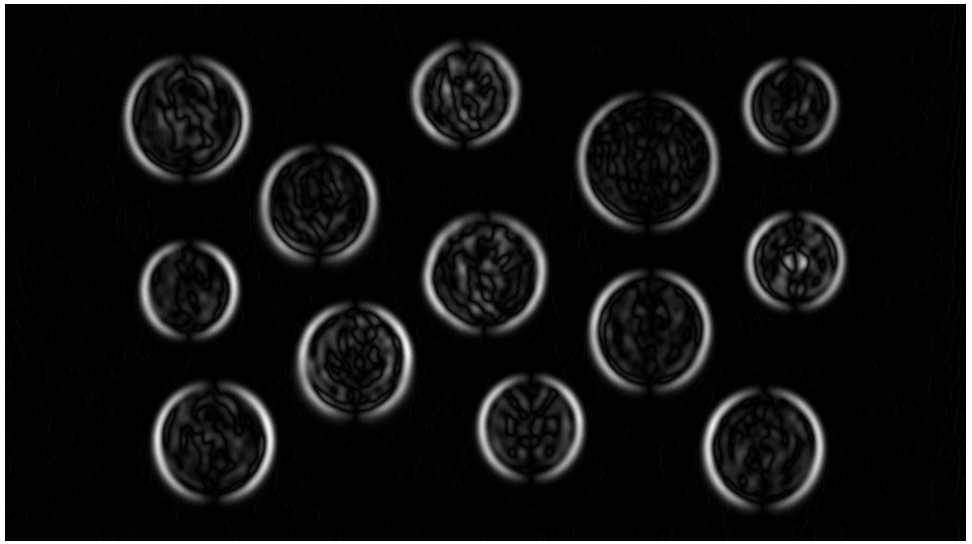
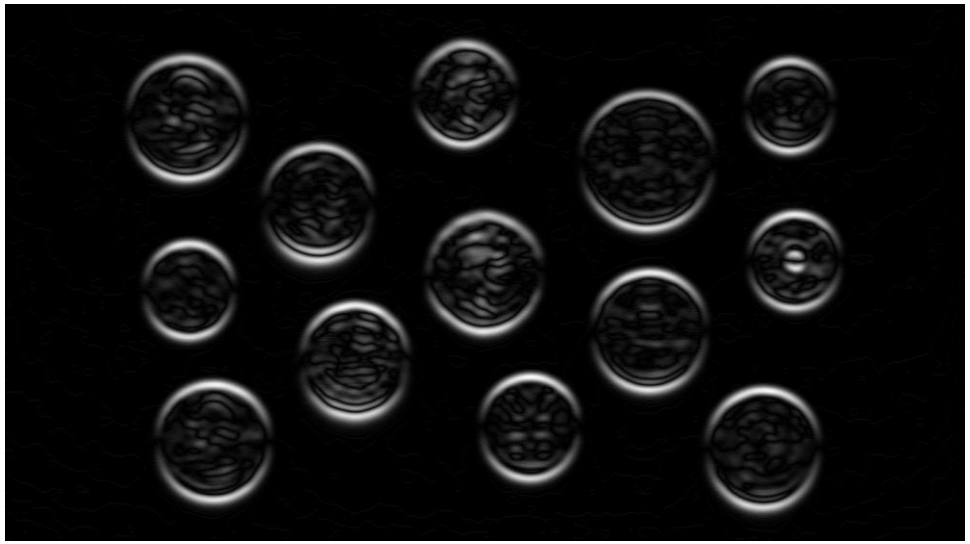
# Gradient magnitude
magnitude = np.sqrt(sobelx**2 + sobely**2)
magnitude = np.uint8(np.clip(magnitude, 0, 255))
```

⚠️ **Important**: Use `cv2.CV_64F`, not `cv2.CV_8U`!

Otherwise negative values get clipped to 0 and you lose half the edges.



Sobel X and Y Results

Sobel X (G_x) — Vertical edges	Sobel Y (G_y) — Horizontal edges
	
Brightness changes left → right	Brightness changes top → bottom



Gradient Magnitude: All Edges Combined

Original	Gradient Magnitude $\left\|\nabla I\right\|$
	
Input image	$\sqrt{G_x^2 + G_y^2}$ — all edge directions combined

Sobel: Limitations

Issue	Description
Thick edges	Not single-pixel precision
Still noise-sensitive	Better than simple diff, but not great
No thresholding	Doesn't tell us *which* pixels are edges

We need a more complete solution...

> "Sobel gives us good gradients, but we need an entire *pipeline* to turn gradients into clean, precise edges. Enter **Canny**."



Other Edge Detection Operators

Prewitt and Laplacian — Broadening Your Toolkit



Prewitt Operator: The Kernels

A Simpler Alternative

$$\text{Prewitt}_X = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} \qquad \text{Prewitt}_Y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

All weights are **uniform** (1, 1, 1) — no center emphasis.



Laplacian: A Different Strategy

2nd-Derivative Approach

- Sobel/Prewitt: **1st derivative** → find where change is greatest (peak)
- Laplacian: **2nd derivative** → find where change-rate crosses zero

$$f''(x) = \lim_{h \to 0} \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}$$

$$\Delta f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

$$f(x+1, y) - 2f(x, y) + f(x-1, y)$$

$$f(x, y+1) - 2f(x, y) + f(x, y-1)$$

$$\Delta f(x, y) \approx f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1) - 4f(x, y)$$

$$4f(x, y) - (\text{sum of 4 neighbors})$$



Laplacian Kernels

Common Discrete Approximations

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix} \quad \text{or} \quad \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

- Left: 4-connected (only direct neighbors)
- Right: 8-connected (includes diagonals)



1st vs 2nd Derivative

Measures **the acceleration of change**. Exactly where the 1st derivative reaches its peak, the 2nd derivative passes through zero. This transition from a positive value to a negative value is called the Zero-crossing.

How They Find Edges Differently

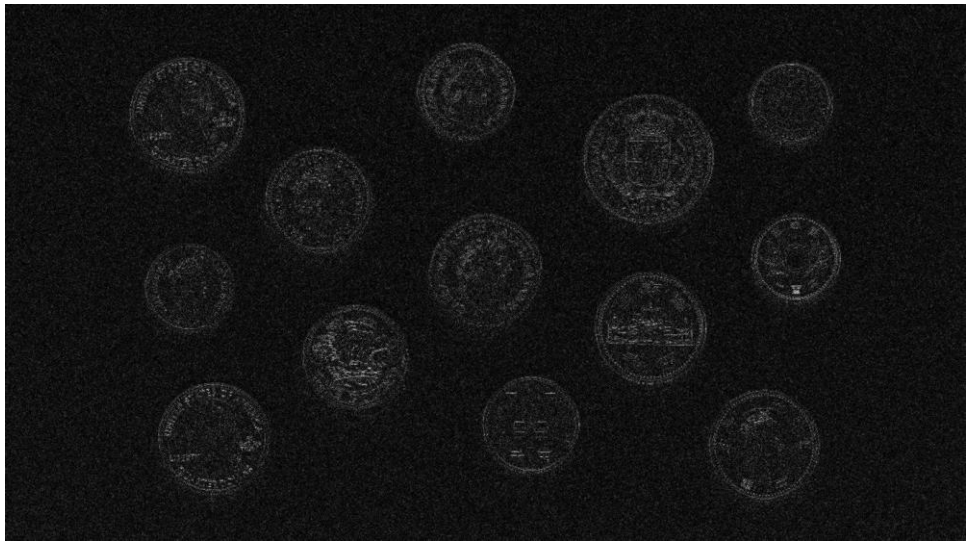
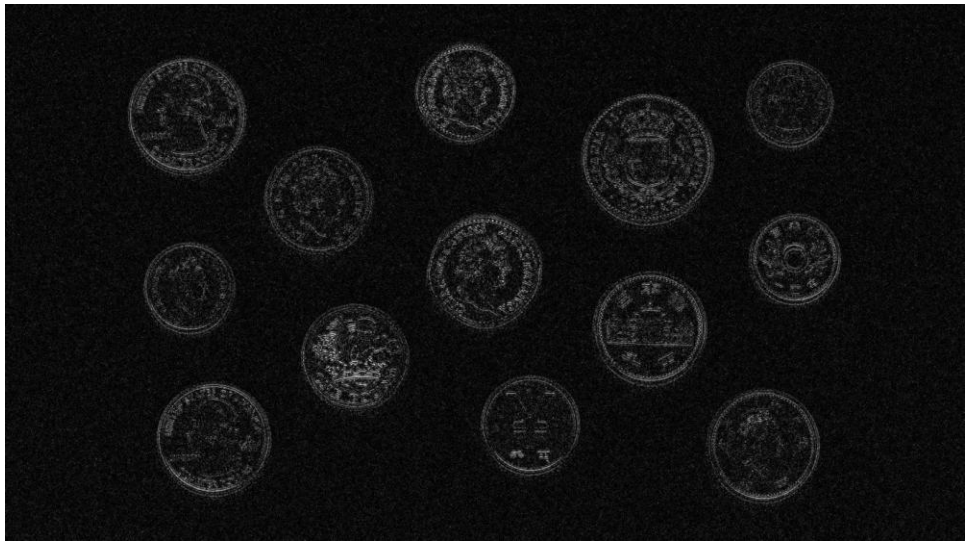
Property	1st Derivative (Sobel)	2nd Derivative (Laplacian)
Edge location	**Peak** of derivative	**Zero-crossing** of 2nd derivative
Direction info	Yes (G_x, G_y)	No (isotropic)
Noise sensitivity	Moderate	**Very high**
Typical use	General edge detection	LoG (with Gaussian pre-smoothing)

Key advantage of Laplacian: Isotropic — detects edges in all directions equally.

Key disadvantage: Extremely sensitive to noise!



Laplacian vs LoG

Direct Laplacian	LoG (Gaussian + Laplacian)
	
Noise overwhelms edges ❌	Smooth first → clean edges ✅

Canny Edge Detection

The Gold Standard — 40 Years and Still Counting



The Gold Standard

A Bit of History

Fact	Detail
Inventor	John Canny (MIT, 1986)
Origin	Master's thesis — later published as a landmark paper
Citations	25,000+ (one of the most cited papers in CV)
Status	Still the default edge detector in OpenCV after **40 years**

Canny's approach: **first define what "optimal" means, then design the algorithm.**



Canny's Three Design Goals

Goal	What It Means	How Canny Achieves It
Low error rate	Detect all real edges, avoid false edges	Gaussian smoothing + dual threshold
Precise localization	Detected edge should be at the true position	Non-Maximum Suppression (NMS)
Single response	One detection per true edge (single-pixel wide)	NMS ensures this

Sobel fails on all three: it's noisy, produces thick edges, and can generate multiple responses per edge.



Five-Step Pipeline — Overview

Step	Operation	Purpose	Depth
1	Gaussian Smoothing	Remove noise	Brief (Week 2)
2	Gradient Computation	Find candidate edges	Brief (just learned Sobel)
3	**Non-Maximum Suppression**	**Thin edges to single pixel**	**Detailed**
4	Dual Threshold	Classify strong vs weak edges	Concept
5	Edge Tracking by Hysteresis	Connect valid weak edges	Concept

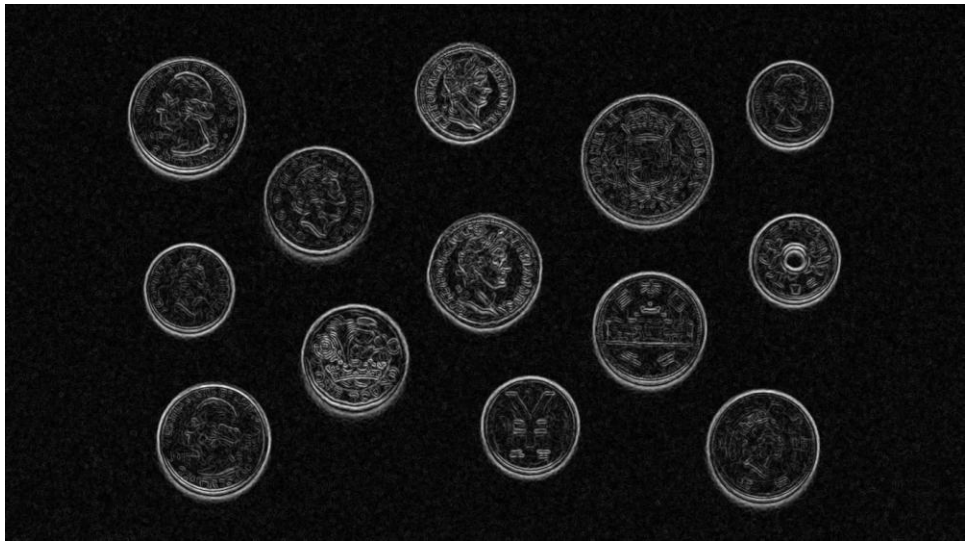
> Image → Smooth → Gradient → NMS → Threshold → Track → Edges



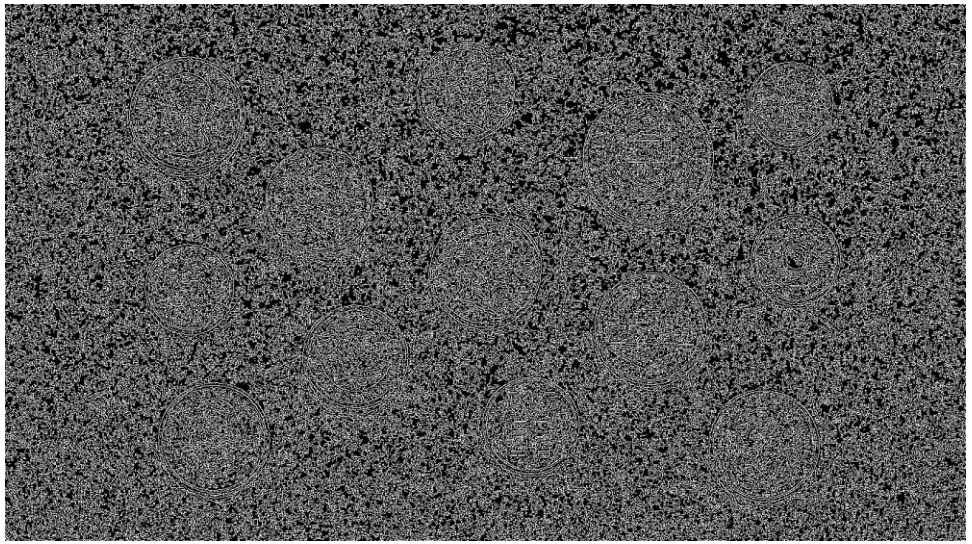
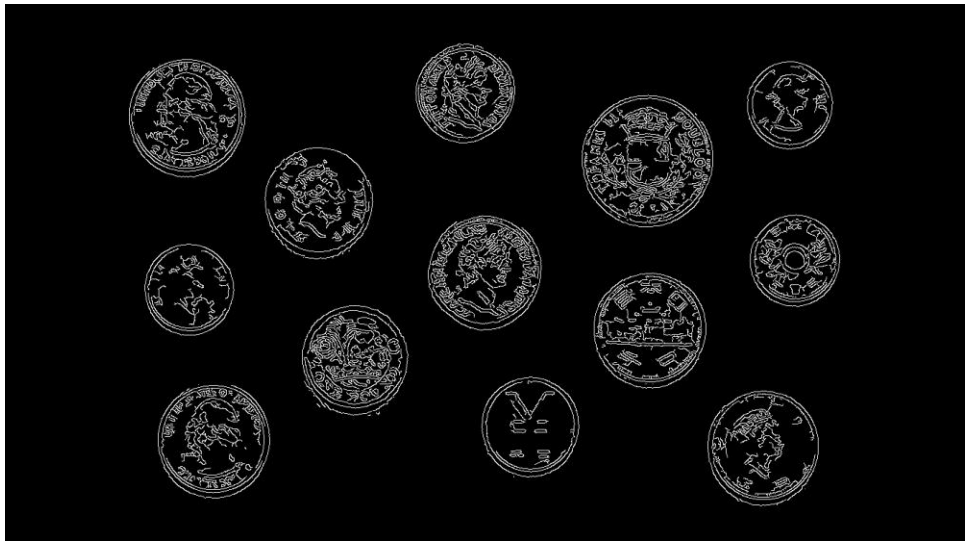
Step 1: Gaussian Smoothing

Noisy Input	After Gaussian Smoothing
	
Random pixel fluctuations → false edges	Noise removed, structures preserved ✓

Step 2: Gradient Computation

Smoothed Image	Gradient Magnitude (Sobel)
	
Input to Sobel	Edges visible but **thick** (5+ pixels wide)

With vs Without Gaussian

Without Gaussian	With Gaussian
	
Noise creates massive false edges ❌	Clean, reliable edges ✅

Step 3: Non-Maximum Suppression (NMS)

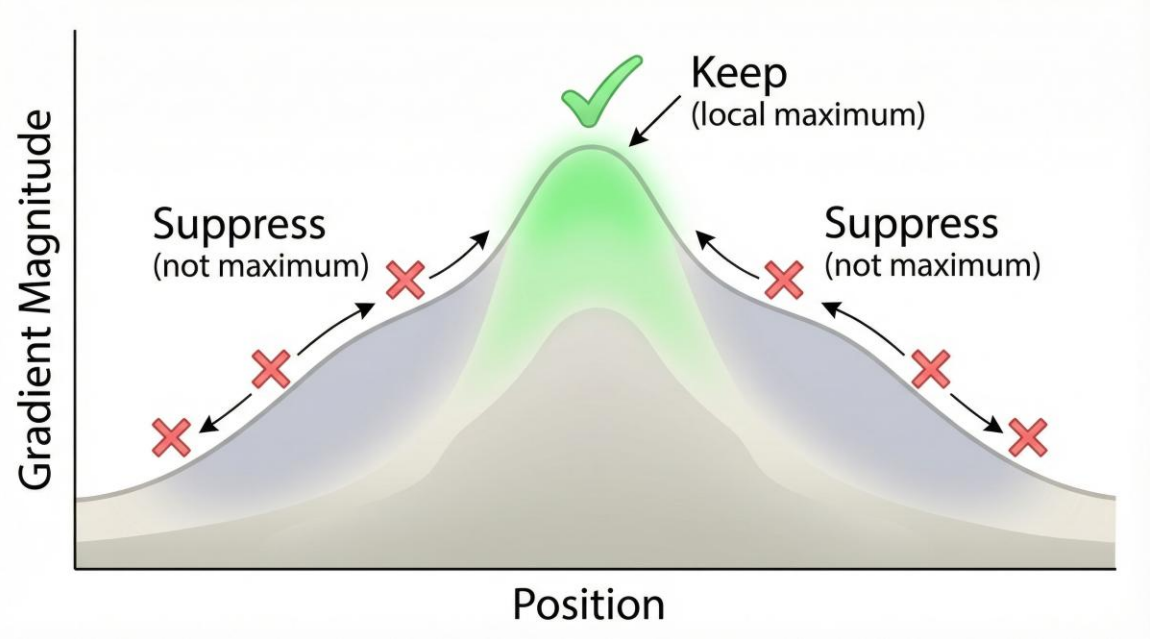
The Core Innovation of Canny

Problem: Edges are **thick bands** (5–10 pixels wide)

Goal: Keep only the **ridge** — the pixel with the largest gradient

Result: Edges become **single-pixel thin**

> Along the gradient direction, keep the **local maximum**, suppress everything else.

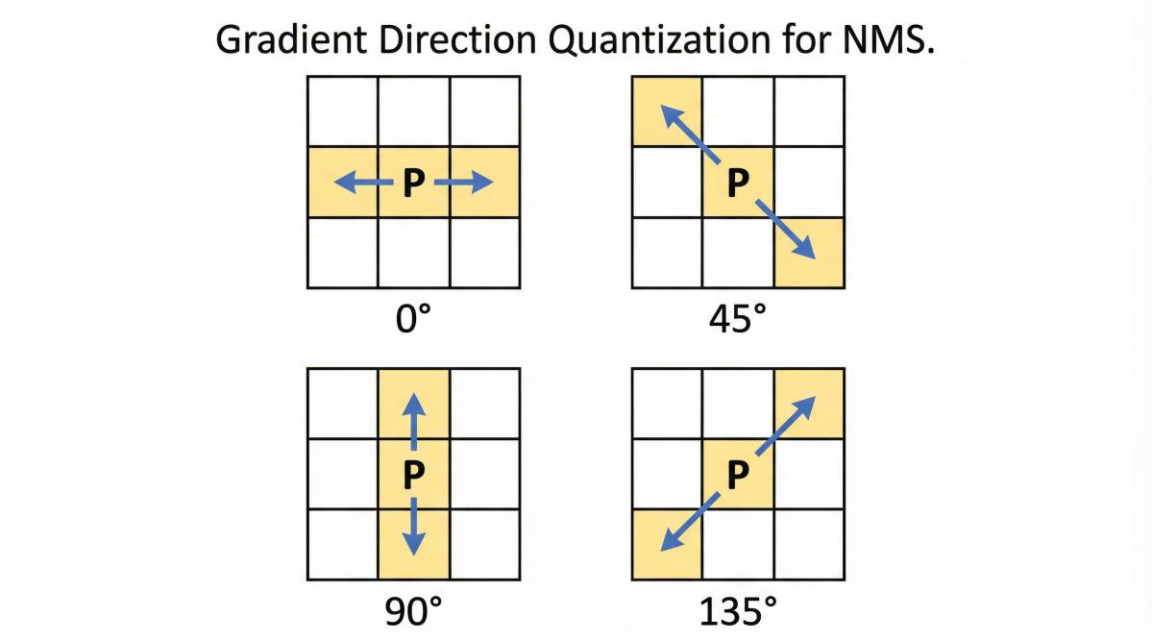


NMS: Direction Quantization

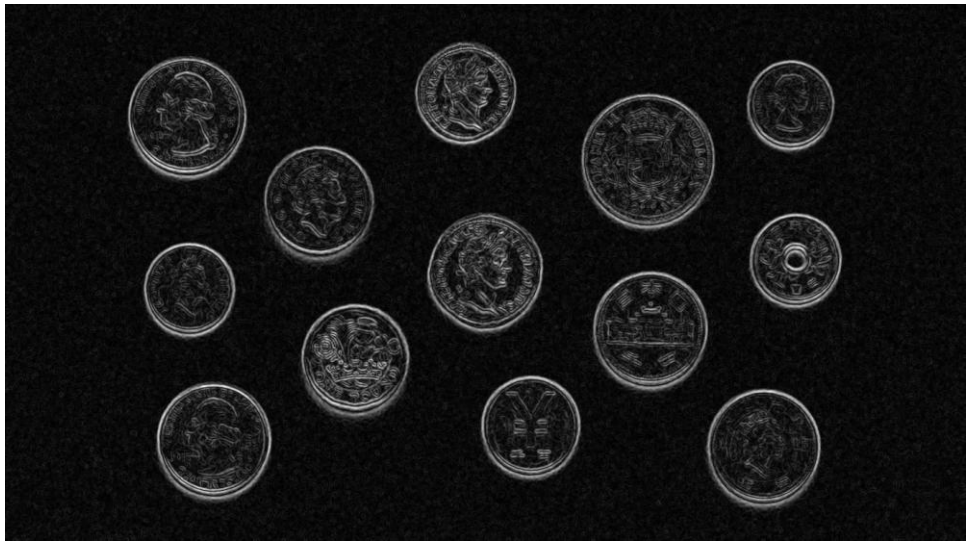
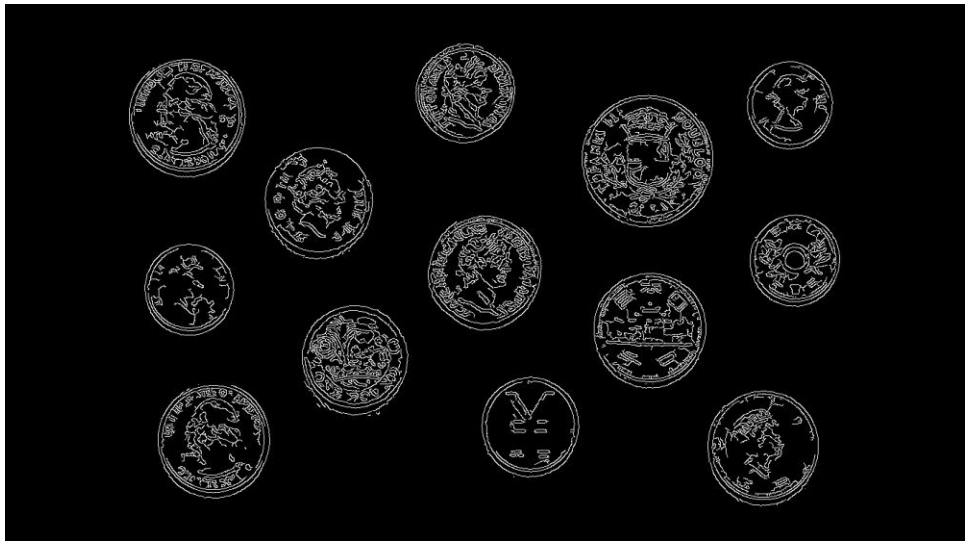
Continuous θ is quantized into **4 directions**:

Angle Range	Direction	Compare With
−22.5°–22.5°	0°	Left & Right
22.5°–67.5°	45°	↖ & ↘
67.5°–112.5°	90°	Top & Bottom
112.5°–157.5°	135°	↗ & ↙

Rule: Current ≥ both neighbors → **keep**; else → **suppress**.



NMS: Before vs After

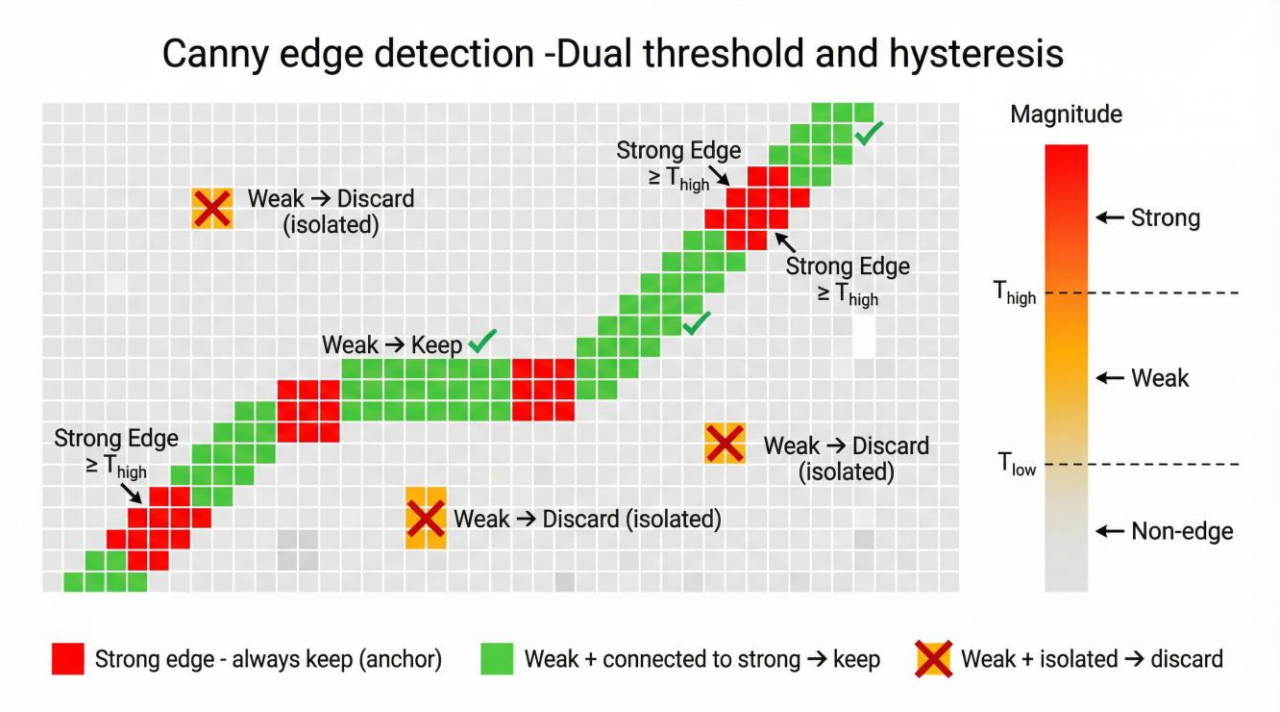
Before NMS (Gradient Magnitude)	After NMS (Canny)
	
Thick edges (5+ pixels wide)	**Single-pixel thin** edges ✓

Step 4: Dual Threshold

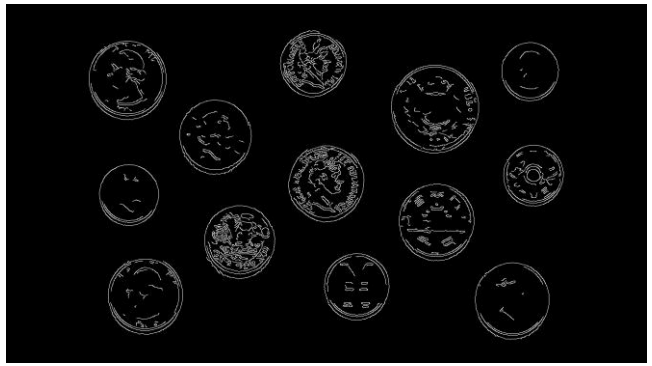
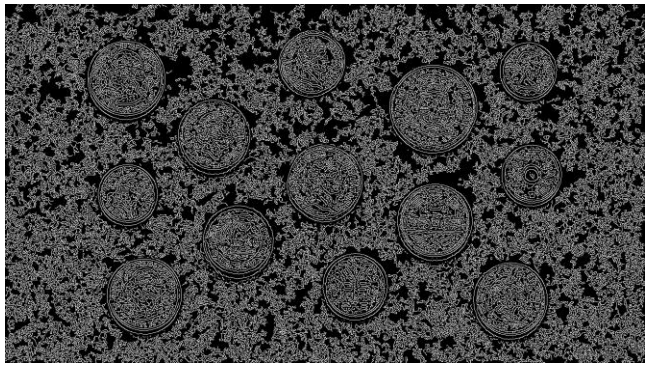
Single threshold: an impossible trade-off

	T too high	T too low
Noise	✓ suppressed	✗ false edges
Continuity	✗ edges break	✓ complete

Solution: Use **two** thresholds.



Threshold Comparison

Too High (100, 200)	Too Low (10, 50)
	
Edges broken, gaps ✗	Too many false edges ✗

Dual Threshold: Three Edge Classes

Classify by Strength

Class	Rule
Strong $\geq T_{high}$	Always keep — **anchor**
Weak $\in [T_{low}, T_{high})$	Keep **only if connected** to strong
Non-edge $< T_{low}$	Always discard

> **Key insight**: strength alone isn't enough — **connectivity** decides.

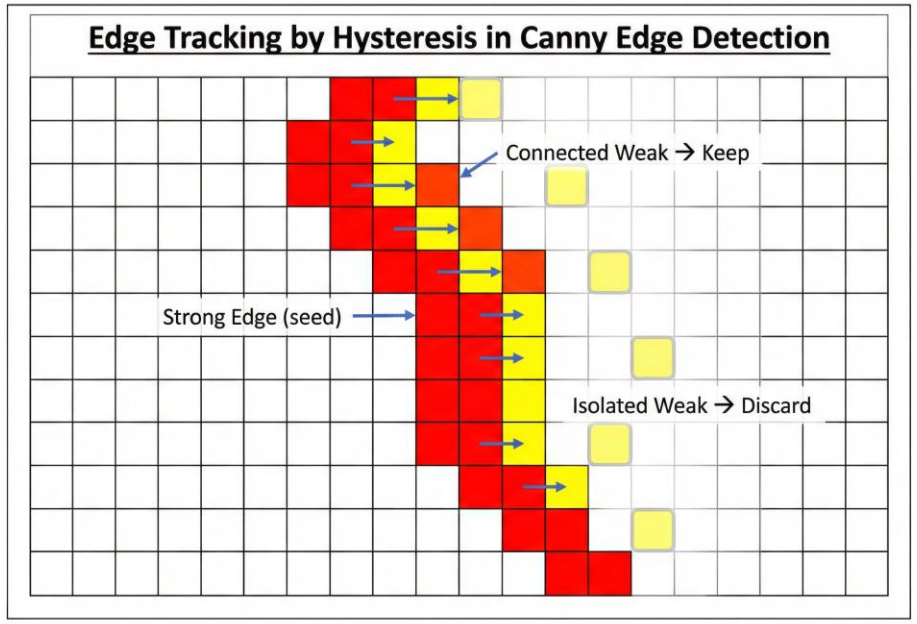


Step 5: Edge Tracking by Hysteresis

Connecting the Dots

After dual thresholding, we have **strong** and **weak** edges:

1. **Start** from each strong edge pixel (seeds)
2. **Search** 8-connected neighbors for weak edges
3. **Promote** connected weak edges → strong
4. **Discard** isolated weak edges (noise)



Strong edges are **seeds**. Weak edges near seeds **grow**. Isolated weak edges die!

Canny Pipeline: Full Visualization

Putting All Five Steps Together

Step	Input → Output	Key Effect
1. Gaussian	Noisy → Smooth	Remove noise
2. Sobel	Smooth → Gradient magnitude	Find candidate edges (thick)
3. NMS	**Thick → Thin edges**	**Precise localization**
4. Dual Threshold	Thin → Strong + Weak	Classify edge confidence
5. Hysteresis	Strong + Weak → Final	Connect real edges

Steps 1–2: Use what we already know (Week 2 + earlier today)

Steps 3–5: Canny's key innovations — making edges precise, clean, and connected



Canny in OpenCV

```
# One line does all 5 steps!
edges = cv2.Canny(img, threshold1=50, threshold2=150)
```

⚠️ Canny's built-in Gaussian uses a fixed 5×5 kernel. For noisy images, apply your own `GaussianBlur` first.



Canny Parameters

Parameter	Description	Typical Values
image	Input grayscale image	—
threshold1	Low threshold T_{low}	50–100
threshold2	High threshold T_{high}	100–200
apertureSize	Sobel kernel size (optional)	3
L2gradient	Use exact L2 norm? (optional)	False

$$Magnitude = \sqrt{G_x^2 + G_y^2} \qquad Magnitude = |G_x| + |G_y|$$



🤔 What If We Skip Steps?

Scenario	What Happens
Skip Step 1 (Gaussian)	Edge map full of noise — false edges everywhere
Skip Step 3 (NMS)	Edges are thick bands — no precise localization
Use single threshold	Either miss edges (too high) or get noise (too low)
Skip Step 5 (Hysteresis)	Weak edges that are part of real edges get discarded



Which Operator Should You Choose?

Scenario	Recommended Operator
General edge detection	**Canny** (built on Sobel)
Need gradient direction	**Sobel**
Isotropic detection needed	**LoG**
Real-time performance	**Sobel** (single-step)

> "In practice, **Canny** handles 90% of cases. But understanding these operators helps you appreciate *why* Canny is so effective."



Why Blur Before Edge Detection?

A Seeming Contradiction



Edge → Contour

Two Concepts, One Pipeline



Edge: A Pixel-Level Concept

What is an Edge?

Property	Description
Nature	Pixel-level label: edge or not-edge
Data structure	2D binary image (H × W)
Continuity	**Not guaranteed** — edges can be broken
Analogy	Scattered ink drops on paper

An edge image is just a collection of pixels — it doesn't know which pixels belong together.



Contour: A Geometric Concept

What is a Contour?

Property	Description
Nature	Geometric/topological: a **curve**
Data structure	Ordered sequence of points: $[(x_1, y_1), (x_2, y_2), \dots]$
Continuity	**Guaranteed** — always a continuous curve, usually closed
Analogy	Lines drawn with a single stroke

A contour knows: "these points form a connected boundary around something."



Edge vs Contour: The Key Differences

Aspect	Edge	Contour
What is it?	Pixel labels	Connected curve
Data format	2D image (array)	List of point sequences
Continuous?	No	Yes
Can count objects?	No (just pixels)	**Yes** (each contour = one boundary)
Can measure shape?	No	**Yes** (area, perimeter, etc.)

> "**Edge** tells you *where* the boundaries are. **Contour** tells you *what shapes* those boundaries form."



The Complete Pipeline

$$\text{Original} \xrightarrow{\text{Canny}} \text{Edge Image} \xrightarrow{\text{findContours}} \text{Contour List}$$

To count coins, we need **contours**, not just edges!

Each contour is an **object boundary** we can count and analyze.



Contour Extraction

From Edge Pixels to Connected Curves



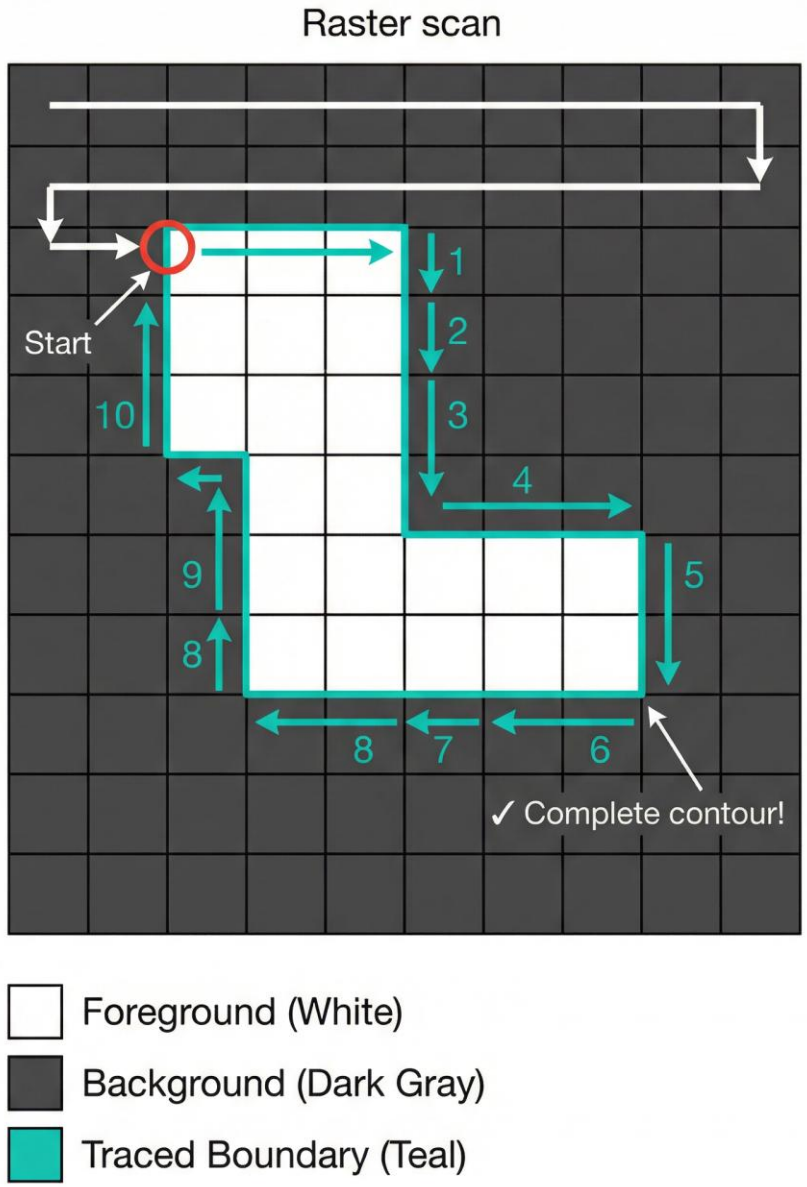
The Core Problem

Connecting Edge Pixels

Given a binary image (white foreground, black background):

1. **Find** where each boundary starts
2. **Trace** along the boundary pixel by pixel
3. **Return** to start → one complete contour
4. **Repeat** for all boundaries

"Blind Person Touching a Wall"



Contour Hierarchy

Parent-Child Relationships

Contours can be **nested**:

Relationship	Meaning
Outer contour	The outermost boundary of an object
Inner contour (hole)	A hole inside an object
Parent	The contour that contains another
Child	A contour contained within another

OpenCV returns hierarchy as: `[Next, Previous, First_Child, Parent]`

Example: A donut has an **outer** contour (circle) and an **inner** contour (hole).



`findContours` API

```
contours, hierarchy = cv2.findContours(
    binary_img,             # Binary image (black bg, white fg)
    cv2.RETR_TREE,          # Retrieval mode
    cv2.CHAIN_APPROX_SIMPLE  # Approximation method
)
```

Returns:

- `contours`: list of contours, each is a numpy array of shape $(N, 1, 2)$
- `hierarchy`: parent-child relationships between contours

⚠️ Input **must** be a binary image (use Canny output or thresholded image).



Retrieval Modes

Mode	What It Returns	When to Use
`RETR_EXTERNAL`	Only the outermost contours	**Simple counting** (coins, cells)
`RETR_LIST`	All contours, no hierarchy	Need all contours, don't care about nesting
`RETR_TREE`	All contours + full hierarchy	**Complex analysis** (objects with holes)

For **coin counting**: use `RETR_EXTERNAL` — we only care about the outer boundaries.



Approximation Methods

How Points Are Stored

Method	Description	Memory
`CHAIN_APPROX_NONE`	Store **every** boundary pixel	Large (a rectangle needs ~400 points)
`CHAIN_APPROX_SIMPLE`	Compress segments into **endpoints only**	Small (a rectangle needs only **4 points**!)

`CHAIN_APPROX_SIMPLE` is almost always the right choice — same accuracy, much less memory.



Contour Properties

What Can We Measure?

Property	API	Use Case
Area	`cv2.contourArea(c)`	Object size
Perimeter	`cv2.arcLength(c, True)`	Boundary length
Bounding Box	`cv2.boundingRect(c)`	Enclosing rectangle
Min. Enclosing Circle	`cv2.minEnclosingCircle(c)`	Circular approximation
Centroid	`cv2.moments(c)` → compute	Object center



Computing the Centroid

```
M = cv2.moments(contour)
cx = int(M["m10"] / M["m00"])  # x-coordinate of center
cy = int(M["m01"] / M["m00"])  # y-coordinate of center
```

- `M["m00"]` = area (zeroth moment)
- `M["m10"] / M["m00"]` = center of mass in x
- `M["m01"] / M["m00"]` = center of mass in y



Contour Filtering

Keeping Only What Matters

```
valid_contours = []
for c in contours:
    area = cv2.contourArea(c)
    if 100 < area < 10000:      # Area range filter
        perimeter = cv2.arcLength(c, True)
        circularity = 4 * np.pi * area / (perimeter ** 2)
        if circularity > 0.7:    # Shape filter
            valid_contours.append(c)
```



Common Filter Criteria

Filter	Purpose	Example
Area range	Remove noise (too small) or background (too large)	`100 < area < 10000`
Circularity	Keep only round objects	`circularity > 0.7`
Aspect ratio	Keep only elongated objects	`0.5 < w/h < 2.0`



Drawing Contours

```
# Draw ALL contours in green, line width 2
cv2.drawContours(img, contours, -1, (0, 255, 0), 2)

# Draw only contour at index 0 in blue
cv2.drawContours(img, contours, 0, (255, 0, 0), 2)

# Draw filled contour (thickness = -1)
cv2.drawContours(img, [contour], -1, (0, 255, 0), -1)
```



`drawContours` Parameters

Parameter	Meaning
`contourIdx = -1`	Draw all contours
`color = (0, 255, 0)`	Green (BGR format)
`thickness = 2`	Line width in pixels
`thickness = -1`	Fill the contour



Part 10

Contour Analysis Toolkit

Shape Descriptors, Convex Hulls, and More



Circularity — How Round Is This Shape?

$$\text{Circularity} = \frac{4\pi \times \text{Area}}{\text{Perimeter}^2}$$

- A perfect circle has circularity = **1.0**
- Lower values indicate more irregular shapes



Circularity of Common Shapes

Shape	Circularity
Perfect circle	**1.0**
Square	≈ 0.785
Rectangle (2:1)	≈ 0.70
Star	≈ 0.3
Irregular	< 0.5



Circularity in Code

```
area = cv2.contourArea(contour)
perimeter = cv2.arcLength(contour, True)
circularity = 4 * np.pi * area / (perimeter ** 2)
```

This is our **key tool** for identifying coins (high circularity).

- `circularity > 0.7` → likely a circle
- `circularity < 0.5` → irregular shape



Aspect Ratio — Width vs Height

Aspect Ratio = w/h — how elongated is the shape?

```
x, y, w, h = cv2.boundingRect(contour)
aspect_ratio = float(w) / h
```

- Square-like objects: ≈ 1.0
- Elongated objects: $\gg 1$ or $\ll 1$
- Useful for distinguishing coins from elongated debris



Extent — How Much of the Bounding Box Is Filled?

Extent = Contour Area / Bounding Box Area

```
extent = float(cv2.contourArea(contour)) / (w * h)
```

- Circle: ≈ 0.785 (π/4)
- Square: = 1.0
- Low extent → irregular shape with lots of concavity



Shape Descriptors: At a Glance

Shape	Circularity	Aspect Ratio	Extent
Circle	≈ 1.0	≈ 1.0	≈ 0.78
Square	≈ 0.78	= 1.0	= 1.0
Rectangle	≈ 0.5–0.7	≠ 1	= 1.0
Star	≈ 0.3	≈ 1.0	≈ 0.4

These descriptors let you **distinguish shapes programmatically** — no deep learning needed!

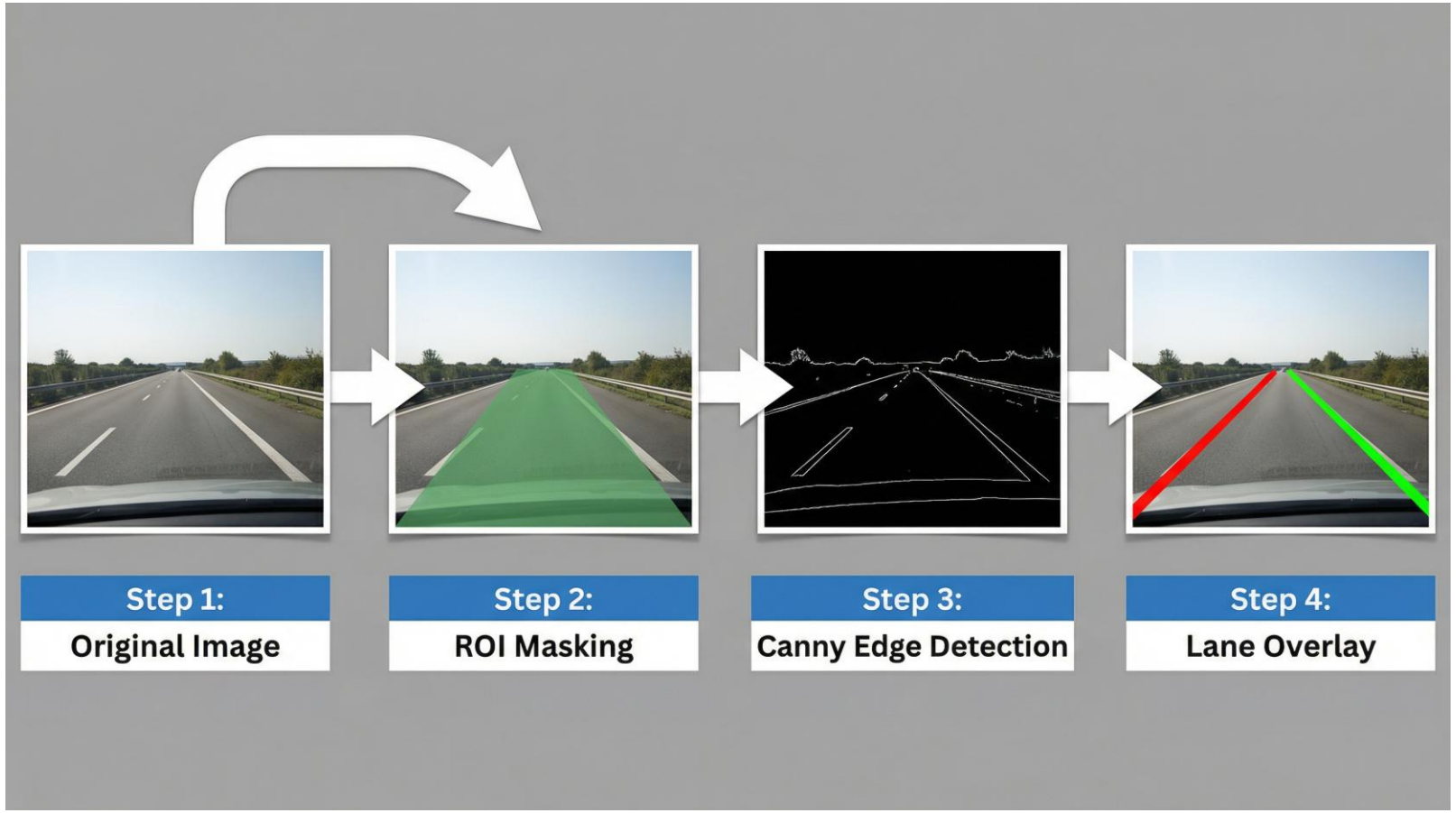


Real-World Applications

Edge Detection Powers Many Technologies



Lane Detection (Autonomous Driving)



Lane Detection (Autonomous Driving)

Step	Operation	What It Does
1	ROI Selection	Focus on the road area (trapezoid mask)
2	Canny Edge Detection	Find lane line edges
3	Hough Transform	Detect straight lines (Week 4+)
4	Line Fitting	Filter and fit lane lines

> "This is how your phone's *lane departure warning* works — Canny edge detection is Step 1."



Medical Image Analysis

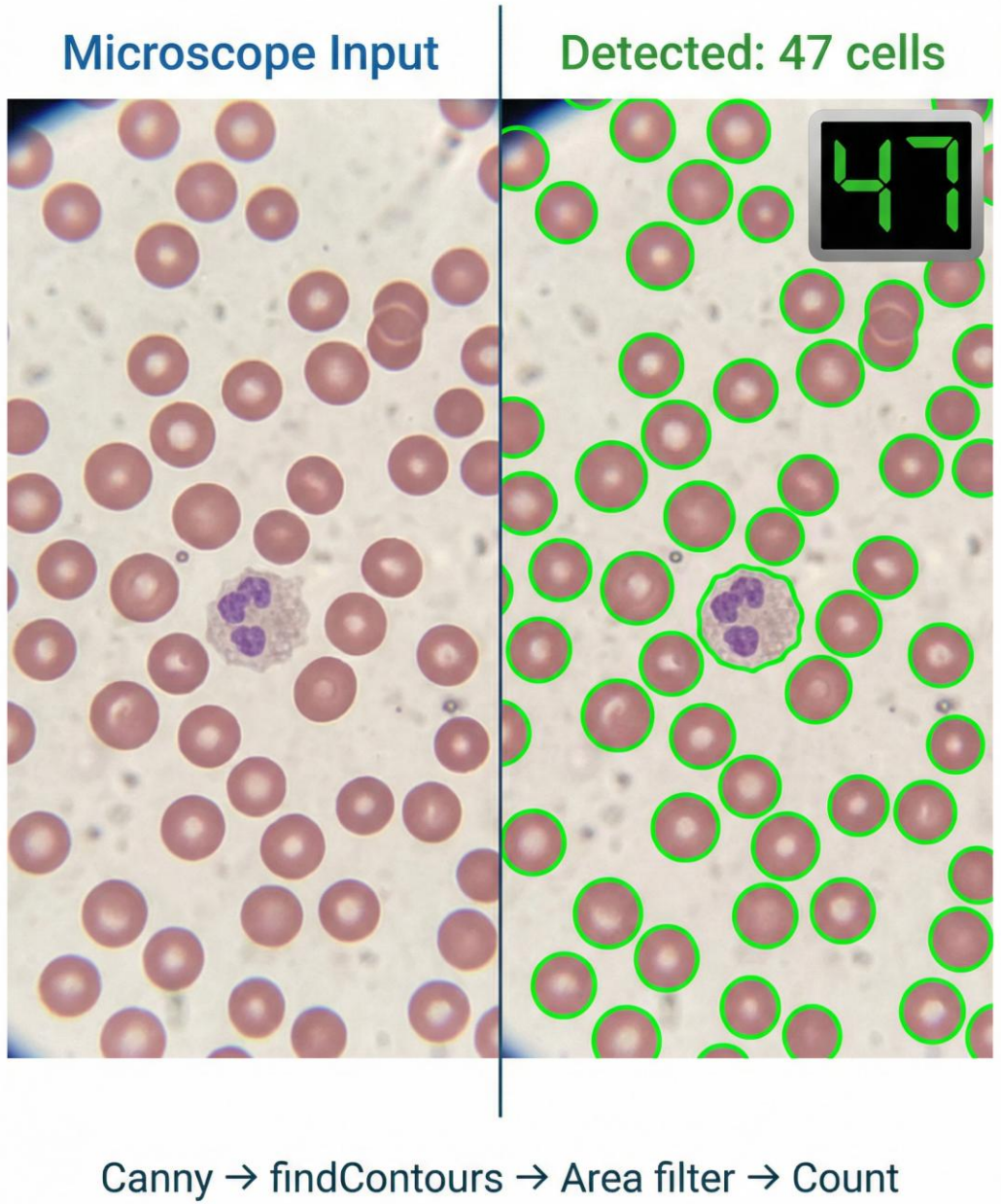
Cell Counting (same idea as coin counting!):

- Microscope → Canny → findContours → count
- Used in blood analysis, cancer research

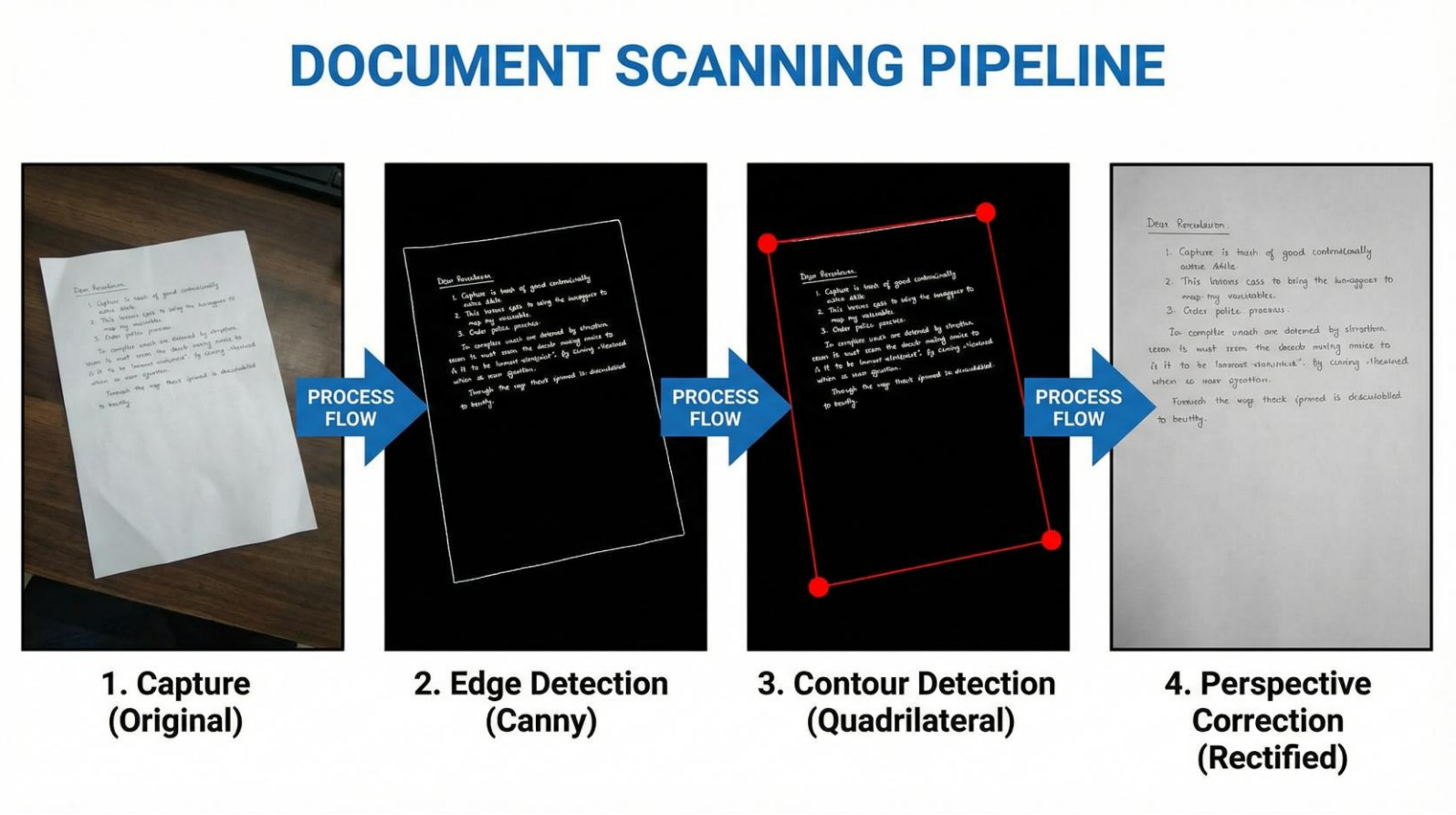
Tumor Boundary Detection:

- CT/MRI → edge detection → contour extraction
- Computer-assisted surgery planning

> "Counting coins today, counting **cells** in research."



Document Scanning



"Every time you scan a document with your phone, edge detection + contour extraction makes it work."

Coin Counting Challenge — Putting It All Together

From Pipeline to Result



The Complete Pipeline

Step	Operation	Code	What It Does
1	Preprocessing	`cvtColor` + `GaussianBlur`	Grayscale + denoise
2	Edge Detection	`cv2.Canny`	Find precise edges
3	Contour Extraction	`cv2.findContours`	Connect edges into curves
4	Contour Filtering	Area + Circularity	Keep only coin-shaped objects
5	Visualization	`cv2.drawContours`	Draw results + count

> **Pipeline**: Coins → Gray+Blur → Canny → findContours → Filter → Count!



Code Walkthrough — Steps 1-3

```
# Step 1: Preprocessing
gray = cv2.cvtColor(coins, cv2.COLOR_BGR2GRAY)
blurred = cv2.GaussianBlur(gray, (5, 5), 1.5)

# Step 2: Edge Detection
edges = cv2.Canny(blurred, 50, 150)

# Step 3: Contour Extraction
contours, _ = cv2.findContours(
   edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
```



Code Walkthrough — Step 4: Filtering

```
valid_coins = []
for c in contours:
    area = cv2.contourArea(c)
    if 500 < area < 50000:
        perim = cv2.arcLength(c, True)
        circ = 4 * np.pi * area / (perim ** 2)
        if circ > 0.7:
            valid_coins.append(c)

print(f"Detected {len(valid_coins)} coins")  # → 12
```



Three Methods — From Failure to Success

Method 1: High-Freq	Method 2: Canny (no filter)	Method 3: Canny + Filter
		
Count: 237 ✗	Count: ~23 ✗	**Count: 12** ✓

Three Methods — Analysis

Method	Edge Source	Filtering	Count	Result
1. High-Freq + Contours	`Original − Blur`	None	237	❌ Fail
2. Canny + Contours	`cv2.Canny`	None	~23	❌ Close but wrong
3. Canny + Smart Filter	`cv2.Canny`	Area + Circularity	**12**	✅ **Success!**

Key improvements:

1. High-Freq → **Canny**: Edge quality dramatically better (today's core lesson)
2. No filter → **Smart filter**: Shape descriptors remove noise contours



Final Result: Count = 12

- **12 coins** detected
- Each outlined in green
- Each numbered at centroid

From "looks like edges" to **precise counting** — the power of Canny + Contours + Shape analysis.



But Wait... Count Again 🤔

How many coins are *really* here?

- Our pipeline says **12**
- But the actual answer is... **13!**
- Top-center: a small Roman silver coin — **completely missed**

> Even a well-designed pipeline can fail on real-world data.



Why Did This Happen?

The 13th Coin — A Tough Case

Property	Most Coins	The Missing Coin
Surface	Smooth, clear relief	Complex, detailed texture
Edge quality	Clean, continuous	Broken, fragmented
Contour	Closed loop → detected	Open fragments → **lost**

The problem: Canny detected edges on the coin's surface texture, but the **outer boundary** was broken into fragments. `RETR_EXTERNAL` couldn't find a single closed contour.

The lesson: Real-world objects don't always play by the rules. Our 5-step pipeline works on 12 out of 13 coins — that's **92% accuracy**. Impressive, but not perfect.



Key Takeaways

1. **Edge = where brightness changes most** — defined by gradient magnitude
2. **Sobel** computes gradients using convolution — connecting Week 2 and Week 3
3. **Canny** is the gold standard: Gaussian → Gradient → **NMS** → Dual Threshold → Hysteresis
4. **NMS** is Canny's core innovation — single-pixel precision
5. **Edge ≠ Contour**: edges are pixel labels, contours are connected curves
6. **Contour analysis** (area, circularity, convex hull) enables smart object detection
7. **Full pipeline**: preprocessing → edge detection → contour extraction → filtering → result
8. **Real world is messy**: even a correct pipeline may miss edge cases — always verify your results!

Anonymous Class Feedback

Ending Quiz



Online Tutorial

https://bloombook.cn/tutorials/af3a31269c3c408fa334e5d515b5b9a0

